SHRI MATA VAISHNO DEVI UNIVERSITY

A Project Report on

End-to-End Design, FPGA Prototyping, and RTL-to-GDSII ASIC Implementation of the SMVDU-AHO-32 32-bit RISC-V Processor

Submitted By

Anupam Sarashwat(23BEC014)

Harsh Mishra(23BEC027)

Om Kumar(23BEC038)

Department of Electronics & Communication Engineering

Shri Mata Vaishno Devi University

Kakryal, Katra, J&K (182320) India

Under the supervision of

Dr. Anil Kumar Bhardwaj

Associate Professor, Department of Electronics & Communication Engineering
SMVDU Katra

Department of Electronics & Communication Engineering
SMVDU, Katra
Jammu & Kashmir – 182320
VIth semester - 2026

SHRI MATA VAISHNO DEVI UNIVERSITY

School of Electronics and Communication Engineering

STUDENT DECLARATION

We hereby declare that the work presented in the B. Tech Mini Project Report entitled **"End-to-End Design, FPGA Prototyping, and RTL-to-GDSII ASIC Implementation of the SMVDU-AHO-32 32-bit RISC-V Processor"** , submitted in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology in Electronics & Communication Engineering** to the **School of Electronics & Communication Engineering, Shri Mata Vaishno Devi University, Katra, J&K**, is an **authentic record of our original work** carried out during the period **August 2025 to December 2025**, under the supervision of **Dr. Anil Kumar Bhardwaj.**

We further declare that the contents of this report are the outcomes of our effort, study, and analysis, and have **not been submitted previously** to any other institute or university for the award of any degree or diploma.

Date: ……...

(signature)

Anupam Sarashwat

23BEC014

…….

(signature)

Harsh Mishra

23BEC027

…….

(signature)

SHRI MATA VAISHNO DEVI UNIVERSITY

School of Electronics and Communication Engineering

CERTIFICATE

This is to certify that the Mini Project entitled "**End-to-End Design, FPGA Prototyping, and RTL-to-GDSII ASIC Implementation of the SMVDU-AHO-32 32-bit RISC-V Processor"**, being submitted by **Anupam Sarashwat (23BEC014), Harsh Mishra (23BEC027)** & **Om Kumar (23BEC038)** to the **School of Electronics and Communication Engineering**, **Shri Mata Vaishno Devi University, Katra, J & K**, has been successfully completed under the supervision and guidance of **Dr. Anil Kumar Bhardwaj**

The contents of this report meet the standards and requirements prescribed in the regulations for the award of the **Bachelor of Technology degree in Electronics & Communication Engineering**.

We extend our best wishes to the students for their future endeavors.

Project Guide

Dr. Anil Kumar Bhardwaj
Associate Professor,
Department of E&CE

…….

(signature)

Head of Department

Dr. Anil Kumar Bhardwaj
Associate Professor,
Department of E&CE

……

(signature)

SHRI MATA VAISHNO DEVI UNIVERSITY

School of Electronics and Communication Engineering

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our project guide, Dr. Anil Bharadhwaj, for his continuous guidance, encouragement, and technical support throughout the development of this project. His valuable suggestions and expertise in digital system design and VLSI implementation greatly helped us during every phase of this work.

We also thank the Department of Electronics and Communication Engineering, Shri Mata Vaishno Devi University, for providing laboratory facilities, FPGA development boards, and access to software tools such as Xilinx Vivado and Cadence EDA tools.

We extend our gratitude to our friends and classmates for their support and motivation during the completion of this project.

Finally, we thank our parents and family members for their constant encouragement and support.

Anupam Sarashwat (23BEC014)
Harsh Mishra (23BEC027)
Om Kumar (23BEC038)

ABSTRACT

The growing demand for open-source processor architectures and custom embedded computing systems has increased the importance of practical processor design and implementation methodologies. This project presents the end-to-end design, FPGA prototyping, and RTL-to-GDSII ASIC implementation of the **SMVDU-AHO-32**, a custom 32-bit RISC-V processor based on the RV32I instruction set architecture.

The processor was designed using synthesizable Verilog HDL and implemented as a single-cycle architecture containing instruction memory, data memory, arithmetic logic unit (ALU), register file, control unit, and immediate generation unit. Functional verification was performed using simulation testbenches to validate arithmetic, logical, memory, branch, and jump instructions.

The RTL design was synthesized and prototyped on a Xilinx Zynq-7000 FPGA using Vivado Design Suite. Hardware testing and waveform verification confirmed successful execution of the implemented instruction set.

To extend the project toward industrial semiconductor design methodologies, the processor was also taken through a complete ASIC RTL-to-GDSII flow using Cadence EDA tools. The frontend flow included logic synthesis, timing analysis, gate-level simulation, logical equivalence checking, and Design for Testability (DFT). The backend flow included floorplanning, placement, clock tree synthesis, routing, static timing analysis, and physical verification.

The final design achieved successful FPGA operation at approximately 65 MHz and generated a DRC-clean and LVS-clean ASIC layout in 180 nm technology. The project demonstrates the complete lifecycle of modern processor design from RTL development to physical chip implementation.

List of Figures

S. No.	Figure No.	Figure Title	Page No.
1	Figure 3.2.1	Block Design of RISC-V	35
2	Figure 3.2.2	RTL Netlist Generated by Vivado EDA	36
3	Figure 3.2.2.1	Program Counter	37
4	Figure 3.2.3.1	Instruction Memory	38
5	Figure 3.2.4.1	Register File: 32-bit	39
6	Figure 3.2.5.1	Immediate Extender	40
7	Figure 3.2.6.1	Arithmetic & Logic Unit	41
8	Figure 3.2.7.1	Control Unit	42
9	Figure 3.2.7.2	Main Decoder	42
10	Figure 3.2.7.3	ALU Decoder	42
11	Figure 3.2.8.1	Data Memory	43
12	Figure 3.2.9.1	Branch and Jump Logic	44
13	Figure 3.2.10.1	Multiplexers in Datapath	44
14	Figure 3.3.1	Simulation Waveform obtained from Testbench	47
15	Figure 3.3.2	Post Simulation Layout	47
16	Figure 3.4.1.1	Top-Level Architecture of PYNQ-Z2 (Zynq-7000 SoC)	46
17	Figure 3.4.1.2	Pinout of PYNQ-Z2	50
18	Figure 3.4.5.1	Program Counter Configured into LEDs	50
19	Figure 3.4.6.1	DRC Report for Design	55

20	Figure 3.4.7.1	On-Chip Power Report	56
21	Figure 4.1.1	Cadence SimVision Console Execution Log	57
22	Figure 4.1.2	Cadence SimVision Waveform Showing Functional Execution	58
23	Figure 4.2.1	Cadence IMC Coverage Analysis – Overall Metrics	59
24	Figure 4.2.2	Cadence IMC Coverage Analysis – Datapath Hierarchy	59
25	Figure 4.3.1	OpenRAM Generated SRAM Macro Layout and Files	60
26	Figure 4.4.1	Synthesis Run 1	61
27	Figure 4.4.2	Cadence Genus Synthesis GUI – Instruction Memory Hierarchy	62
28	Figure 4.4.3	Cadence Genus Synthesis GUI – Data Memory Hierarchy	62
29	Figure 4.4.4	Cadence Genus Synthesis GUI – Core Block Hierarchy	62
30	Figure 4.4.5	Cadence Genus Synthesis GUI – Top SoC Module Bifurcation	63
31	Figure 4.5.1	SimVision GLS vs Golden RTL Output Verification	65
32	Figure 4.5.2	SimVision GLS vs Golden RTL Output Verification 2	65
33	Figure 4.5.3	SimVision GLS vs Golden RTL Output Verification 3	65
34	Figure 4.5.4	SimVision GLS vs Golden RTL Output Verification 4	65
35	Figure 5.1.1	Floorplan of Core Design	71
36	Figure 5.1.2	Power Rails	72
37	Figure 5.1.3	Power Rings	72
38	Figure 5.1.4	IO Pin Placement	73
39	Figure 5.1.5	Standard Cell Placement	73
40	Figure 5.1.6	Clock Tree Synthesis (CTS)	74

41	Figure 5.1.7	Post Route View	75
42	Figure 5.2.1	SRAM Macro Integration into SoC Floorplan	75
43	Figure 5.2.2	SRAM Placement and Routing	75
44	Figure 5.3.1	SoC Floorplanning	77
45	Figure 5.3.2	Top-Level Power Grid	78
46	Figure 5.3.3	Initial Pin Placement	78
47	Figure 5.3.4	Optimized Pin Placement	79
48	Figure 5.3.5	Legalized Pin Placement	79
49	Figure 5.3.6	Pre-Route RC Check	79
50	Figure 5.3.7	Top-Level Placement and Routing	80
51	Figure 5.3.8	Physical Design Completed	80

List of Tables

Table 3.1.1: Processor Specifications..... 30
Table 3.1.2: Supported RV32I Base Integer Instruction Formats..... 31
Table 3.4.2.1: Major features of PYNQ-Z2..... 51
Table 4.4.1: Comparative Analysis……...63
Table 5.1.1.1: Final Post Route…….75
Table 5.1.2.1: Routing & DRC…….....76
Table 5.4.3.1:Delay Breakdown..... 82
Table 5.4.3.2:Post-Layout ASIC Implementation Results Summary..... 83

List of Symbols

S. No.	Symbol	Description
1	ISA	Instruction Set Architecture
2	RV32I	32-bit Base Integer RISC-V Instruction Set
3	RTL	Register Transfer Level
4	HDL	Hardware Description Language
5	FPGA	Field Programmable Gate Array
6	ASIC	Application Specific Integrated Circuit
7	SoC	System on Chip
8	ALU	Arithmetic Logic Unit
9	PC	Program Counter
10	GPR	General Purpose Register
11	SRAM	Static Random Access Memory
12	BRAM	Block Random Access Memory
13	CTS	Clock Tree Synthesis
14	STA	Static Timing Analysis
15	DRC	Design Rule Check
16	LVS	Layout Versus Schematic
17	GDSII	Graphic Database System II Layout File
18	DFT	Design for Testability
19	LUT	Look-Up Table
20	FF	Flip-Flop
21	GPIO	General Purpose Input Output
22	UART	Universal Asynchronous Receiver Transmitter
23	SPI	Serial Peripheral Interface
24	XDC	Xilinx Design Constraints
25	EDA	Electronic Design Automation
26	CMOS	Complementary Metal Oxide Semiconductor

27	PPA	Power, Performance, and Area
28	WNS	Worst Negative Slack
29	TNS	Total Negative Slack
30	IR Drop	Voltage Drop due to Interconnect Resistance
31	IPC	Instructions Per Cycle
32	CPI	Cycles Per Instruction
33	Tc	Clock Period
34	VDD	Power Supply Voltage
35	VSS	Ground Reference Voltage
36	LEF	Library Exchange Format
37	LIB	Timing Library File
38	SDC	Synopsys Design Constraints
39	RC	Resistance-Capacitance
40	PLL	Phase Locked Loop
41	QoR	Quality of Results
42	IMC	Integrated Metrics Center
43	GLS	Gate-Level Simulation
44	OpenRAM	Open-Source SRAM Compiler
45	Verilog	Hardware Description and Modeling Language
46	RV64I	64-bit Base Integer RISC-V ISA
47	MHz	Megahertz
48	μm^2	Square Micrometre
49	ns	Nanosecond
50	mW	Milliwatt

Contents

STUDENT DECLARATION ii
CERTIFICATE iii
(signature) iii
ACKNOWLEDGEMENT iv
ABSTRACT v
List of Figures vi
List of Tables ix
List of Symbols x
Contents xii
Chapter 1: Introduction 15
1.1 Introduction to Microprocessors 15
1.2 Introduction to RISC-V Architecture 16
1.3 Motivation of the Project 18
1.4 Problem Statement 19
1.5 Objectives of the Project 20
1.6 Scope of the Work 20
Chapter 2: Literature Review 22
2.1 Processor Architectures 22
2.2 FPGA Prototyping Techniques 23
2.3 ASIC Design Methodologies 24
2.4 RTL-to-GDSII Design Flow 25
2.5 Performance Analysis 26
2.6 Related Research Work 28
CHAPTER 3 - PHASE 1: ARCHITECTURAL DESIGN & FPGA VERIFICATION 30
3.1 Processor Microarchitecture and Instruction Set Architecture (ISA): 30
3.2 Architectural Design 35
3.3 Verification Strategy 45
3.4 FPGA Prototyping and Hardware Verification 48
CHAPTER 4 - PHASE 2 (PART I): ASIC FRONTEND FLOW 57
4.1 Functional Verification (Cadence IRun) 57
..... 58
4.2 Code Coverage Analysis 58
4.3 SRAM Macro Generation (OpenRAM) 59

4.4 Synthesis & Timing Constraints (Iterative Analysis) 60
4.5 Gate Level Simulation (GLS) & LEC 64
4.6 Design for Testability (DFT) and Scan Chain Insertion 66
CHAPTER 5 - PHASE 2 (PART II): ASIC BACKEND & SoC INTEGRATION 71
5.1 Core Physical Design 71
5.2 SoC Top Wrapper Design & Integration 76
5.3 Top-Level Physical Implementation 77
5.4 Post-Route ASIC Analysis — SMVDU-AH032 SoC 81
CHAPTER 6 - RESULTS AND ANALYSIS 86
6.1 RTL Verification & Coverage Summary 86
6.2 FPGA Performance Results 86
6.3 ASIC Synthesis Metrics (Frontend) 87
6.4 GLS and DFT Results 87
6.5 Timing Reports and Final Layout Analysis (Backend) 88
CHAPTER 7 – CHALLENGES FACED 89
7.1 Managing Routing Congestion Around OpenRAM 89
7.2 Timing Closure (Hold Violations) 89
7.3 Resolving Specific DRC Violations 89
7.4 FPGA Hardware Debugging 89
7.5 Limitations in DFT and ATPG Implementation 90
CHAPTER 8 – APPLICATIONS AND BENEFITS TO SOCIETY 91
8.1 Applications of the SMVDU-AHO-32 Processor 91
8.2 Benefits to Society 92
CHAPTER 9 – CONCLUSION AND FUTURE SCOPE 94
9.1 Conclusion 94
9.2 Future Scope 95
REFERENCES 97

Chapter 1: Introduction

1.1 Introduction to Microprocessors

The relentless scaling of semiconductor technologies, historically governed by Moore's Law and Dennard Scaling, has catalyzed the widespread integration of embedded microprocessors across diverse computational domains. Continuous advancements in Very Large-Scale Integration (VLSI) technology have enabled the development of compact, high-performance, and energy-efficient computing systems that power modern electronic devices. From consumer electronics and industrial automation to communication systems, automotive platforms, healthcare devices, and Internet of Things (IoT) applications, embedded processors have become indispensable components of contemporary digital infrastructure.

Modern System-on-Chip (SoC) architectures demand highly optimized, power-efficient, scalable, and application-specific processor cores capable of meeting stringent performance and energy constraints. While general-purpose processors are suitable for heterogeneous computational workloads, customized silicon implementations are increasingly preferred for deterministic, low-latency, and energy-constrained embedded systems. The growing demand for domain-specific architectures has therefore accelerated research in customizable processor microarchitectures and open hardware ecosystems.

Traditional commercial processor architectures such as x86 and ARM are proprietary and require licensing agreements for implementation and deployment. These restrictions often limit architectural flexibility and accessibility for academic institutions, startups, and independent hardware developers. To address these challenges, the RISC-V instruction set architecture emerged as a modern open-source alternative that enables unrestricted processor design, customization, and extension without royalty or licensing limitations.

RISC-V, developed at the University of California, Berkeley, follows the Reduced Instruction Set Computing (RISC) philosophy emphasizing simplified instruction execution, modularity, extensibility, and efficient hardware implementation. Its clean-slate architecture, fixed-length instruction encoding, and modular extension methodology make it highly suitable for embedded processor design, FPGA prototyping, and ASIC implementation workflows. Consequently, RISC-V has rapidly gained popularity in both academia and semiconductor industries for research, education, and commercial hardware development.

In this project, a custom 32-bit RISC-V processor named SMVDU-AHO-32 was designed and implemented using synthesizable Verilog Hardware Description Language (HDL). The processor is based on the RV32I base integer instruction set architecture and follows a single-cycle datapath implementation methodology. The design was initially verified through RTL simulation and subsequently prototyped on the PYNQ-Z2 FPGA platform using the Vivado Design Suite for synthesis, implementation, timing analysis, and hardware validation.

Following successful FPGA prototyping, the processor was further taken through a complete RTL-to-GDSII ASIC implementation flow using industrial Electronic Design Automation (EDA) tools. The frontend flow included RTL verification, logic synthesis, gate-level simulation, Design for Testability (DFT), and Logical Equivalence Checking (LEC), while the backend physical design flow included floorplanning, power planning, placement, Clock Tree Synthesis (CTS), routing, Static Timing Analysis (STA), Design Rule Check (DRC), Layout Versus Schematic (LVS) verification, and final GDSII generation. The project therefore demonstrates a complete end-to-end processor implementation methodology spanning RTL development, FPGA prototyping, and full-custom ASIC realization.

1.2 Introduction to RISC-V Architecture

The evolution of computer architecture has been significantly influenced by the distinction between Complex Instruction Set Computing (CISC) and Reduced Instruction Set Computing (RISC) philosophies. Early processor architectures mainly followed the CISC approach, which emphasized large and complex instruction sets capable of performing multiple low-level operations within a single instruction. Although CISC architectures reduced software complexity, they introduced increased hardware complexity, complicated instruction decoding, higher power consumption, and difficulties in achieving efficient pipelined execution.

To overcome these limitations, the Reduced Instruction Set Computing (RISC) architecture emerged as an alternative processor design philosophy emphasizing simplified instructions, faster execution, efficient hardware implementation, and optimized performance. RISC architectures employ a load-store paradigm in which memory access operations are separated from arithmetic and logical operations. Only load and store instructions access memory, while arithmetic and logical computations are performed exclusively on processor registers. This simplifies datapath organization, reduces hardware overhead, and improves execution efficiency.

RISC architectures also utilize simplified instruction decoding, fixed-length instruction formats, and a smaller set of highly optimized instructions. These characteristics enable efficient pipelining, reduced control logic complexity, lower silicon area utilization, and improved performance-per-watt. Consequently, RISC processors became highly suitable for embedded systems, communication devices, mobile platforms, and energy-constrained applications.

Historically, proprietary RISC architectures such as ARM, MIPS, SPARC, and PowerPC dominated the embedded processor industry. However, despite their technical advantages, these architecture imposed licensing costs and limited architectural flexibility for researchers, startups, and educational institutions. This created the need for an open and flexible instruction set architecture that could support both academic research and industrial innovation without commercial restrictions.

RISC-V emerged as a solution to these challenges. RISC-V is an open-source Instruction Set Architecture (ISA) based on the principles of Reduced Instruction Set Computing. It was developed in 2010 at the University of California, Berkeley, by a research team led by Professor Krste Asanović and Professor David Patterson. The primary objective behind the development of RISC-V was to create a modern, simple, modular, scalable, and extensible processor architecture that could be freely used for academic research, industrial development, and hardware innovation without licensing restrictions.

The name “RISC-V” represents the fifth major generation of RISC-based processor research carried out at UC Berkeley. Unlike proprietary processor architectures, RISC-V is openly available under permissive licensing models, allowing developers and organizations to design, modify, extend, and manufacture processors without paying royalties. This open-source nature has made RISC-V one of the most important developments in modern computer architecture and semiconductor engineering.

RISC-V follows the fundamental RISC philosophy emphasizing simplified instruction execution, fixed-length instruction encoding, efficient hardware implementation, and modular architecture organization. The architecture is specifically designed to minimize hardware complexity while maintaining high computational performance, scalability, and flexibility. Due to its clean and modular structure, RISC-V simplifies processor implementation and enables easier understanding of processor microarchitecture concepts such as datapath design,

instruction decoding, control signal generation, memory organization, branch handling, and pipelined execution.

One of the most important strengths of RISC-V is its modular architecture design. The ISA consists of a compact base instruction set along with optional standard and custom extensions that can be integrated according to application requirements. This modularity allows hardware engineers to design lightweight embedded processors, high-performance application processors, and domain-specific accelerators optimized for precise Power, Performance, and Area (PPA) requirements.

The architecture supports multiple implementation widths including 32-bit, 64-bit, and 128-bit variants, enabling scalability across different computational domains ranging from low-power embedded systems to advanced server-class processors. Due to its extensibility and flexibility, RISC-V has gained widespread adoption in embedded systems, FPGA prototyping platforms, artificial intelligence accelerators, Internet of Things (IoT) devices, educational processor design, and industrial ASIC development workflows.

The simplicity and openness of RISC-V also make it highly suitable for academic learning and VLSI research. Researchers and students can study processor microarchitecture, instruction execution, datapath organization, control path generation, memory hierarchy, timing optimization, and physical chip implementation without proprietary limitations. Consequently, RISC-V has rapidly gained popularity among universities, research laboratories, semiconductor startups, and major technology companies worldwide.

1.3 Motivation of the Project

A significant gap exists between academic digital design education and the practical implementation methodologies used in the semiconductor industry. Most academic processor design projects are generally limited to RTL coding, simulation, or FPGA prototyping. While these stages are important for understanding digital system design, they do not fully expose students to the complexity of modern ASIC implementation and physical chip design.

In industrial VLSI design, processor implementation extends far beyond RTL verification. Modern ASIC development involves synthesis optimization, timing closure, clock distribution, power planning, routing, Static Timing Analysis (STA), and strict sign-off verification procedures such as Design Rule Check (DRC) and Layout Versus Schematic (LVS) verification. These stages are essential for ensuring manufacturability, timing correctness, reliability, and power efficiency.

Another important motivation behind this project is the growing demand for engineers skilled in both FPGA prototyping and ASIC implementation methodologies. Semiconductor industries increasingly require practical knowledge of the complete digital design flow ranging from RTL development to physical layout generation.

The emergence of the open-source RISC-V architecture further motivated this project by providing a flexible and modular platform for custom processor development without proprietary licensing restrictions. RISC-V enables researchers and students to study processor microarchitecture, datapath organization, instruction decoding, and hardware implementation techniques in detail.

The primary motivation of the SMVDU-AHO-32 project is therefore to bridge the gap between academic processor design and industrial semiconductor implementation practices. The project demonstrates the complete processor development lifecycle starting from Verilog RTL design and FPGA prototyping on the PYNQ-Z2 platform to a complete RTL-to-GDSII ASIC implementation flow using Cadence EDA tools. Through this project, practical exposure to frontend and backend VLSI methodologies, timing optimization, physical verification, and ASIC design challenges was achieved.

1.4 Problem Statement

The objective of this project is to design, verify, and implement a fully functional 32-bit RISC-V processor named SMVDU-AHO-32 based on the RV32I base integer instruction set architecture. The processor is required to correctly execute arithmetic, logical, branch, jump, and memory access operations while maintaining efficient datapath and control path functionality.

The project involves translating a behavioural Verilog RTL specification into a physically realizable hardware implementation through a two-stage validation methodology. The first stage focuses on functional verification and hardware validation through FPGA prototyping on the PYNQ-Z2 platform using the Vivado Design Suite. This stage includes RTL simulation, synthesis, timing analysis, and real-time hardware testing.

The second stage involves performing a complete RTL-to-GDSII ASIC implementation flow using Cadence EDA tools. This includes logic synthesis, Design for Testability (DFT), floorplanning, placement, Clock Tree Synthesis (CTS), routing, Static Timing Analysis (STA), Design Rule Check (DRC), Layout Versus Schematic (LVS) verification, and final GDSII generation targeted toward a sub-micron semiconductor technology node.

The project therefore aims to bridge the gap between academic RTL design and industrial ASIC implementation methodologies by demonstrating the complete lifecycle of processor development from behavioural HDL design to physical chip realization.

1.5 Objectives of the Project

The major objectives of the SMVDU-AHO-32 project are as follows:

1. **Microarchitectural Design**:
   To design and implement a fully functional 32-bit RISC-V processor based on the RV32I instruction set architecture using synthesizable and modular Verilog Hardware Description Language (HDL).
2. **Functional Verification and FPGA Prototyping**:
   To verify the functional correctness of the processor through RTL simulation and implement the design on the PYNQ-Z2 based on the Xilinx Zynq-7000 SoC platform for real-time hardware validation and timing analysis.
3. **Frontend Logic Synthesis:**
   To perform ASIC frontend implementation using Cadence Genus for logic synthesis, technology mapping, timing optimization, and power-performance-area (PPA) analysis using appropriate Synopsys Design Constraints (SDC).
4. **Design for Testability (DFT):**
   To incorporate Design for Testability methodologies such as scan-chain insertion and test infrastructure generation to improve fault observability and manufacturing test coverage.
5. **Physical Implementation:**
   To execute the complete ASIC backend physical design flow using Cadence Innovus, including floorplanning, power planning, standard-cell placement, Clock Tree Synthesis (CTS), routing, and timing optimization.
6. **Physical Verification and Sign-off:**
   To achieve timing closure and generate a manufacturable ASIC layout by performing Static Timing Analysis (STA), Design Rule Check (DRC), Layout Versus Schematic (LVS) verification, and final GDSII generation.

1.6 Scope of the Work

The scope of this project is limited to the design and implementation of a 32-bit single-cycle RISC-V processor based on the RV32I base integer instruction set architecture. The processor

follows an unpipelined datapath design methodology to maintain architectural simplicity and to focus primarily on the complete FPGA prototyping and ASIC implementation flow.

The project includes the design and verification of major processor components such as the Program Counter, Register File, Arithmetic Logic Unit (ALU), Control Unit, Immediate Generator, Instruction Memory, Data Memory, and branch/jump control logic using synthesizable Verilog HDL.

The scope further includes:

- RTL simulation and functional verification
- FPGA prototyping on the PYNQ-Z2 platform
- Logic synthesis using Cadence Genus
- Gate-level simulation and Logical Equivalence Checking (LEC)
- Design for Testability (DFT)
- ASIC backend physical design using Cadence Innovus
- Floorplanning, placement, Clock Tree Synthesis (CTS), routing, and Static Timing Analysis (STA)
- Physical verification including Design Rule Check (DRC) and Layout Versus Schematic (LVS) verification
- Final GDSII layout generation

To maintain focus on the frontend and backend ASIC implementation methodology, advanced architectural features such as pipelining, superscalar execution, branch prediction, cache memory hierarchy, multiplication/division extensions, and floating-point operations are excluded from the project scope.

Additionally, analog and mixed-signal verification of SRAM memory macros is outside the scope of this work. Pre-compiled standard-cell libraries and OpenRAM-generated memory compiler outputs such as LEF and LIB files are used as technology-provided black-box components during ASIC implementation.

Chapter 2: Literature Review

2.1 Processor Architectures

The evolution of microprocessor architecture has been one of the most important developments in modern digital system design and VLSI engineering. Early processor architectures mainly focused on maximizing computational capability through increasingly complex instruction sets and hardware functionalities. Over time, the demand for improved performance, lower power consumption, scalability, and efficient hardware implementation led to the emergence of Reduced Instruction Set Computing (RISC) architectures.

Historically, proprietary processor architectures such as ARM, MIPS, SPARC, and PowerPC dominated the embedded systems and System-on-Chip (SoC) industry. Architectures like the ARM Cortex-M series became widely used in low-power embedded applications due to their energy efficiency and optimized hardware implementation. Similarly, MIPS32 architectures played a significant role in academic processor research because of their simple pipeline structure and clean instruction organization.

Although these architectures greatly influenced processor design methodologies, most remained proprietary and required commercial licensing agreements for implementation and customization. This limited architectural flexibility for researchers, educational institutions, and semiconductor startups interested in custom processor development and experimental hardware acceleration techniques.

A major breakthrough in processor architecture research occurred with the introduction of the RISC-V Instruction Set Architecture by David Patterson, Krste Asanović, and their research team at the University of California, Berkeley. RISC-V was proposed as a clean-slate, open-source, modular, and extensible ISA designed to eliminate legacy architectural complexity and licensing restrictions associated with proprietary processor architectures.

The RV32I base integer instruction set forms the foundation of the RISC-V architecture and has become widely adopted for academic processor research and custom hardware accelerator development. The architecture uses fixed-length 32-bit instruction encoding, enabling simplified instruction decoding, efficient datapath implementation, and reduced hardware complexity. Its modular organization also allows designers to include optional extensions according to application requirements, making it highly suitable for embedded systems, FPGA prototyping, artificial intelligence accelerators, and ASIC implementations.

Due to its simplicity, scalability, extensibility, and open-source accessibility, RISC-V has rapidly gained popularity in both academia and semiconductor industries. It has emerged as a preferred architecture for studying processor microarchitecture, instruction execution, datapath organization, control signal generation, timing optimization, and physical ASIC implementation methodologies.

2.2 FPGA Prototyping Techniques

Prototyping digital systems on Field Programmable Gate Arrays (FPGAs) prior to ASIC fabrication has become an essential practice in modern VLSI and semiconductor design methodologies. FPGA prototyping enables designers to validate hardware functionality, timing behavior, and system-level operation in real-time before proceeding to costly ASIC manufacturing processes. Since ASIC fabrication involves significant development cost and time, identifying design errors during the pre-silicon stage is extremely important for reducing overall project risk.

Unlike software-based simulation environments such as ModelSim or Cadence Xcelium, FPGA emulation provides an accurate hardware-level representation of concurrent digital circuit operation. This enables execution of long test sequences, real-time debugging, and validation of processor behavior under realistic operating conditions. FPGA-based testing also allows verification of instruction execution, memory operations, clock behavior, and datapath functionality at near-hardware speeds.

Research in modern processor and SoC design methodologies highlights that FPGA prototyping significantly reduces ASIC respin risks by identifying corner-case functional bugs, timing issues, and hardware integration problems before GDSII generation and fabrication. FPGA implementation therefore acts as an important intermediate validation stage between RTL simulation and ASIC physical design.

In this project, the SMVDU-AHO-32 processor was prototyped on the PYNQ-Z2 based on the Xilinx Zynq-7000 SoC platform. The FPGA implementation was performed using Vivado Design Suite, which was used for synthesis, implementation, timing analysis, bitstream generation, and hardware validation. The FPGA prototyping stage enabled real-time verification of processor functionality, instruction execution, and timing behavior prior to ASIC implementation.

2.3 ASIC Design Methodologies

Modern semiconductor design fundamentally relies on structured ASIC implementation methodologies to achieve reliable, manufacturable, and high-performance integrated circuits. ASIC design methodologies define the sequence of frontend and backend implementation stages required to transform a behavioural hardware description into a physically realizable silicon layout. These methodologies are essential for handling the increasing complexity of modern digital systems and deep-submicron fabrication technologies.

Historically, ASIC implementation followed two primary approaches: full-custom design and standard-cell-based design. Full-custom ASIC design provides maximum optimization in terms of performance, area, and power consumption because every transistor and layout structure is manually designed. However, full-custom implementation requires extensive design effort, longer development cycles, and very high engineering complexity, making it suitable mainly for highly optimized commercial processors and memory circuits.

To overcome these limitations, standard-cell methodology became the dominant approach in modern digital IC design. As described in classical VLSI literature by Jan Rabaey and Neil Weste, standard-cell-based ASIC design utilizes pre-characterized logic cells stored in technology libraries. These libraries contain timing, power, and physical layout information for commonly used digital components such as logic gates, flip-flops, multiplexers, and buffers.

The standard-cell methodology significantly reduces design complexity and development time by automating technology mapping, placement, routing, and timing optimization processes using Electronic Design Automation (EDA) tools. This methodology provides improved design predictability, faster turnaround time, scalability, and efficient post-layout timing analysis while maintaining acceptable Power, Performance, and Area (PPA) characteristics.

Modern ASIC implementation flows are generally divided into frontend and backend stages. Frontend design includes RTL coding, functional verification, logic synthesis, gate-level simulation, and Design for Testability (DFT). Backend physical design includes floorplanning, power planning, placement, Clock Tree Synthesis (CTS), routing, Static Timing Analysis (STA), and physical verification such as Design Rule Check (DRC) and Layout Versus Schematic (LVS) verification.

In this project, the SMVDU-AHO-32 processor follows a standard-cell ASIC implementation methodology using Cadence EDA tools. The design flow automates synthesis, technology mapping, placement, and routing while enabling predictable post-layout timing extraction and

physical verification. This methodology provides practical exposure to industrial semiconductor design workflows and modern ASIC implementation techniques.

2.4 RTL-to-GDSII Design Flow

The RTL-to-GDSII design flow represents one of the most important methodologies in modern VLSI and ASIC engineering. It defines the complete sequence of frontend and backend implementation stages required to transform a behavioural Register Transfer Level (RTL) hardware description into a manufacturable physical chip layout. Modern semiconductor industries extensively utilize automated Electronic Design Automation (EDA) tools to handle the increasing complexity of deep-submicron integrated circuit design.

The frontend stage of the RTL-to-GDSII flow begins with RTL design and functional verification using Hardware Description Languages such as Verilog HDL. After successful simulation and verification, logic synthesis is performed using synthesis tools such as Cadence Genus. During synthesis, abstract Boolean logic is mapped into technology-specific standard-cell gate libraries while optimizing timing, power consumption, and silicon area.

The synthesis process attempts to balance dynamic power consumption and timing constraints. Dynamic switching power in CMOS circuits is generally represented by:

$$P_{dynamic} = \alpha C V^2 f$$

where:

- α represents switching activity,
- C represents load capacitance,
- V represents supply voltage,
- and f represents operating frequency.

Similarly, timing optimization is governed by setup timing constraints represented as:

$$t_{clk} \geq t_{pcq} + t_{pd} + t_{setup} + t_{skew}$$

where:

- t_{pcq} is clock-to-Q delay,
- t_{pd} is combinational propagation delay,

- t_{setup} is setup time,
- and t_{skew} is clock skew.

After logic synthesis, the design enters the backend physical implementation stage using tools such as Cadence Innovus. Backend implementation includes floorplanning, power planning, standard-cell placement, Clock Tree Synthesis (CTS), routing, parasitic extraction, and Static Timing Analysis (STA).

Modern backend literature heavily emphasizes physical implementation challenges such as routing congestion, clock skew minimization, signal integrity preservation, and power distribution reliability. Power grid design is particularly important for mitigating voltage drop effects caused by:

V = IR

where excessive IR drop can degrade circuit reliability and timing performance.

Clock Tree Synthesis focuses on constructing balanced and symmetric clock distribution networks to minimize clock skew and ensure reliable synchronous operation across the chip. Routing algorithms further optimize wire length, congestion, and timing while satisfying manufacturing design rules.

Finally, physical verification stages such as Design Rule Check (DRC) and Layout Versus Schematic (LVS) verification ensure that the generated layout is both manufacturable and electrically equivalent to the synthesized netlist. After successful verification and timing closure, the final physical layout is exported as a GDSII database used for semiconductor fabrication.

In this project, the SMVDU-AHO-32 processor was implemented through a complete Cadence-based RTL-to-GDSII ASIC flow, providing practical exposure to industrial frontend and backend VLSI implementation methodologies.

2.5 Performance Analysis

The SMVDU-AHO-32 processor follows a single-cycle microarchitecture in which every instruction completes execution within one clock cycle. In a single-cycle processor, the clock period must be sufficiently large to accommodate the worst-case combinational propagation delay through the datapath. Consequently, the maximum operating frequency is determined by the critical path delay of the processor.

Processor performance is generally evaluated using execution time, which depends on three major factors:

$$Execution\ Time = Instructions \times CPI \times Clock\ Period$$

where:

- the number of instructions depends on the executed program,
- CPI (Cycles Per Instruction) represents the average number of clock cycles required per instruction,
- and clock period is determined by the processor critical path delay.

In the SMVDU-AHO-32 architecture, the CPI is ideally equal to 1 because each instruction is executed in a single clock cycle. However, the single-cycle implementation introduces a relatively larger clock period since all instruction stages including instruction fetch, decode, register access, ALU execution, memory access, and write-back must complete within the same cycle.

The critical timing path of the processor generally traverses through:

- Program Counter,
- Instruction Memory,
- Register File,
- ALU,
- Data Memory,
- and Write-Back Multiplexer.

The maximum operating frequency of the processor is therefore constrained by the propagation delay of this longest combinational path.

Although single-cycle processors are not optimized for high-frequency operation, they offer several important advantages including:

- simplified control logic,
- easier datapath implementation,
- deterministic execution behavior,

- simplified verification,
- and suitability for FPGA prototyping and educational processor design.

The processor was successfully implemented on the PYNQ-Z2 platform and achieved stable operation at approximately 65 MHz after synthesis and timing optimization using Vivado Design Suite

2.6 Related Research Work

Recent advancements in open-source hardware research have demonstrated the growing viability of custom RISC-V processor implementations for both academic and industrial applications. Several academic and industrial projects have successfully implemented RISC-V-based processors and System-on-Chip (SoC) architectures targeting embedded systems, low-power computing, and hardware accelerator applications.

One of the most influential open-source RISC-V projects is the **Berkeley Rocket Chip** developed at the University of California, Berkeley. The Rocket Chip framework introduced a scalable and configurable RISC-V processor generation methodology capable of supporting multicore architectures, cache hierarchies, and advanced memory systems. Similarly, low-power open-source processor platforms such as **PULPino** demonstrated the effectiveness of lightweight RISC-V cores for energy-efficient embedded and IoT applications.

These projects established the practicality of open-source silicon development and significantly accelerated research in customizable processor architectures and open hardware ecosystems. They also highlighted the advantages of modular instruction set architectures, FPGA prototyping, and scalable SoC development methodologies.

However, many academic publications and open-source processor projects primarily focus on RTL functionality, architectural performance, or FPGA implementation while providing limited discussion on ASIC backend physical design challenges. Important practical implementation aspects such as floorplanning constraints, routing congestion, clock tree optimization, power integrity, SRAM macro integration, timing closure, and physical sign-off verification are often not explored in sufficient detail.

Another critical challenge in modern ASIC implementation involves integrating synthesized standard-cell logic with memory compiler-generated SRAM macros. Standard-cell and memory-macro co-design introduces additional complexities in floorplanning, placement

optimization, routing congestion management, clock distribution, and timing closure due to the physical dimensions and placement constraints associated with SRAM blocks.

The SMVDU-AHO-32 project distinguishes itself by not only implementing a functional RV32I-based RISC-V processor but also transparently documenting the complete RTL-to-GDSII ASIC implementation methodology. The project specifically emphasizes frontend and backend implementation challenges including OpenRAM-generated SRAM macro integration, standard-cell and macro co-design considerations, timing optimization, physical verification, and final ASIC sign-off procedures using Cadence EDA tools. This provides a more practical and industry-oriented perspective on modern processor implementation workflows beyond conventional RTL simulation and FPGA prototyping approaches.

CHAPTER 3 - PHASE 1: ARCHITECTURAL DESIGN & FPGA VERIFICATION

3.1 Processor Microarchitecture and Instruction Set Architecture (ISA):

The SMVDU-AHO-32 microprocessor is a fully synchronous, purely digital 32-bit Harvard architecture processor designed to execute the RISC-V Base Integer Instruction Set Architecture (RV32I). The processor follows a single-cycle datapath implementation methodology in which each instruction completes execution within one clock cycle. The architecture utilizes separate instruction and data memory interfaces, enabling simultaneous instruction fetch and data access operations while simplifying memory organization and datapath control.

The processor was implemented using synthesizable IEEE 1364-2005 Verilog Hardware Description Language (HDL) and designed following modular RTL development practices to ensure portability across FPGA and ASIC implementation flows. The complete architecture consists of multiple interconnected hardware modules including the Program Counter (PC), Instruction Memory, Register File, Arithmetic Logic Unit (ALU), Immediate Generation Unit, Main Control Unit, ALU Decoder, Branch Logic, and Data Memory.

The datapath is controlled by a combinational Control Unit that decodes instruction fields and generates the required control signals for arithmetic execution, memory access, register write-back, branching, and jump operations. The processor implements a load-store architecture in which arithmetic and logical operations are performed exclusively on register operands while dedicated load/store instructions handle memory access operations.

The architecture supports the complete RV32I base integer instruction set using fixed-length 32-bit instruction encoding, simplifying instruction fetch, decode, and hardware implementation. The processor operates within a 32-bit physical address space capable of addressing up to 4 GB of memory.

Table 3.1.1: Processor Specifications

Parameter	***Specification***
Architecture Type	32-bit Load/Store RISC-V Processor
ISA Supported	RV32I Base Integer Instruction Set
Datapath Type	Single-Cycle Datapath

Memory Organization	Harvard Architecture
Address Space	32-bit Physical Addressing (4 GB)
Register File	32 × 32-bit General Purpose Registers
Register x0	Hardwired to Logic 0
Hardware Description Language	Synthesizable IEEE 1364-2005 Verilog HDL
FPGA Platform	PYNQ-Z2
FPGA Device	Xilinx Zynq-7000 xc7z020
ASIC Design Flow	Cadence RTL-to-GDSII Flow

The register file contains thirty-two 32-bit General Purpose Registers (GPRs) with two asynchronous read ports and one synchronous write port. Register x0 is permanently hardwired to logic zero in compliance with the RISC-V ISA specification. The Program Counter increments sequentially by 4 during normal instruction execution and updates to branch or jump target addresses during control transfer operations.

The processor utilizes a single clock domain with an active-high asynchronous reset signal to initialize the Program Counter and sequential storage elements during startup. The architecture was initially verified through RTL simulation and later validated on FPGA hardware before proceeding to full ASIC implementation.

The SMVDU-AHO-32 processor is based on the RV32I Base Integer Instruction Set Architecture of RISC-V. The RV32I ISA utilizes a highly regular and fixed-length 32-bit instruction encoding scheme, significantly simplifying instruction fetch, decode, and control signal generation. The uniform instruction width reduces hardware complexity and enables efficient implementation of the datapath and instruction decoding logic.

Each instruction is composed of multiple fields including opcode, source registers (*rs1, rs2*), destination register (*rd*), function codes (*funct3, funct7*), and immediate fields. These fields are decoded by the Control Unit and ALU Decoder to generate the required datapath control signals for arithmetic operations, memory access, branching, and register write-back.

The processor implements the fundamental RV32I instruction formats including Register-type, Immediate-type, Store-type, Branch-type, Upper Immediate-type, and Jump-type instructions.

Table 3.1.2: Supported RV32I Base Integer Instruction Formats

Format	**Description**	**Instructions Implemented**

R-Type	Register-to-register arithmetic and logical operations	ADD, SUB, SLL, SLT, XOR, SRL, SRA, OR, AND
I-Type	Immediate arithmetic and load operations	ADDI, SLTI, XORI, ORI, ANDI, SLLI, LW
S-Type	Store operations	SW, SH, SB
B-Type	Conditional branch operations	BEQ, BNE, BLT, BGE, BLTU, BGEU
U-Type	Upper immediate operations	LUI, AUIPC
J-Type	Unconditional jump operations	JAL, JALR

The R-Type instructions operate exclusively on register operands and are primarily used for arithmetic and logical computations. I-Type instructions utilize immediate operands for arithmetic calculations and memory load operations. S-Type instructions are responsible for writing data from registers into memory locations. B-Type instructions implement Program Counter-relative conditional branching using comparison results generated by the ALU. U-Type instructions support upper immediate loading and PC-relative address generation, while J-Type instructions perform unconditional control transfer and subroutine call operations.

The fixed-length instruction encoding and modular ISA organization of RV32I significantly simplify processor implementation while maintaining flexibility for future architectural extensions and enhancements.

3.1.1 Instruction Encoding:

R-Type Instruction Format:

R-Type instructions are used for register-to-register arithmetic and logical operations.

funct7	**rs2**	**rs1**	**funct3**	**rd**	**opcode**
7 bits	**5 bits**	**5 bits**	**3 bits**	**5 bits**	**7 bits**

Supported Instructions

- ADD
- SUB

- SLL
- SLT
- XOR
- SRL
- SRA
- OR
- AND

I-Type Instruction Format:

I-Type instructions are used for immediate arithmetic operations, load instructions, and jump-register operations.

immediate[11:0]	**rs1**	**funct3**	**rd**	**opcode**
12 bits	**5 bits**	**3 bits**	**5 bits**	**7 bits**

Supported Instructions

- ADDI
- SLTI
- XORI
- ORI
- ANDI
- SLLI
- LW
- JALR

S-Type Instruction Format:

S-Type instructions are used for memory store operations.

imm[11:5]	**rs2**	**rs1**	**funct3**	**imm[4:0]**	**opcode**

7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

Supported Instructions

- SW
- SH
- SB

B-Type Instruction Format:

B-Type instructions perform conditional Program Counter-relative branching.

imm[12]	imm[10:5]	rs2	rs1	funct3	imm[4:1]	imm[11]	opcode
1 bit	6 bits	5 bits	5 bits	3 bits	4 bits	1 bit	7 bits

Supported Instructions

- BEQ
- BNE
- BLT
- BGE
- BLTU
- BGEU

U-Type Instruction Format:

immediate[31:12]	rd	opcode
20 bits	5 bits	7 bits

Supported Instructions

- LUI
- AUIPC

J-Type Instruction Format:

J-Type instructions are used for unconditional jump operations.

imm[20]	imm[10:1]	imm[11]	imm[19:12]	rd	opcode
1 bit	10 bits	1 bit	8 bits	5 bits	7 bits

Supported Instructions

- JAL

3.2 Architectural Design

The architectural design of the SMVDU-AHO-32 processor was developed using a modular single-cycle datapath methodology based on the RV32I RISC-V instruction set architecture. The processor consists of multiple interconnected hardware modules responsible for instruction fetch, decoding, arithmetic execution, memory access, control signal generation, and register write-back operations.

The architecture follows a Harvard memory organization with separate instruction and data memories, enabling independent instruction fetch and data access operations. All hardware modules were implemented using synthesizable Verilog HDL and designed to ensure compatibility across FPGA and ASIC implementation flows.

The complete processor datapath was divided into several functional blocks including the Program Counter, Instruction Memory, Register File, Immediate Generation Unit, Arithmetic Logic Unit (ALU), Control Unit, Data Memory, branch logic, and multiplexing network. Each hardware block was individually designed and verified before full processor integration.

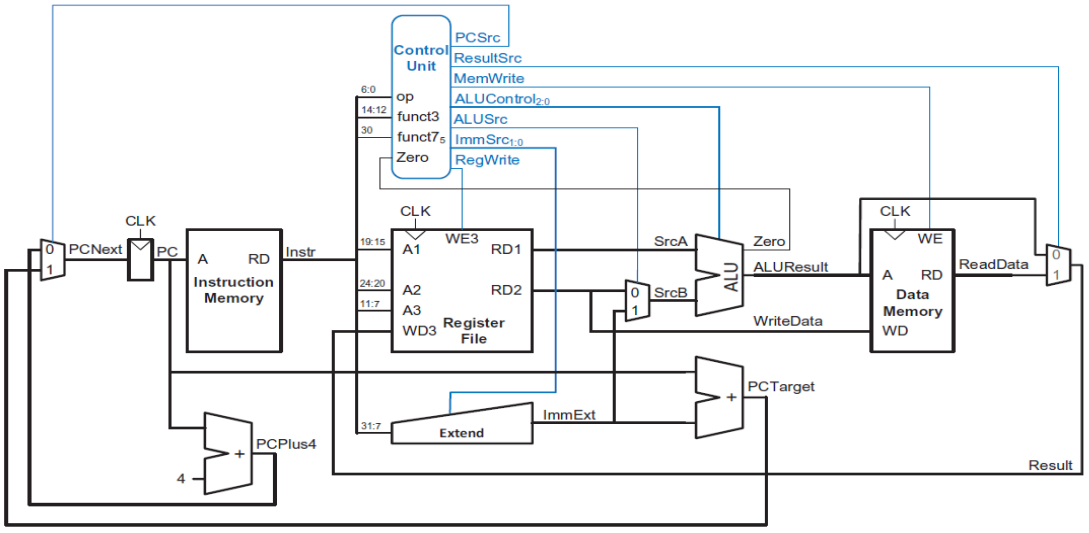


Figure 3.2.1: Block Design of RISC-V

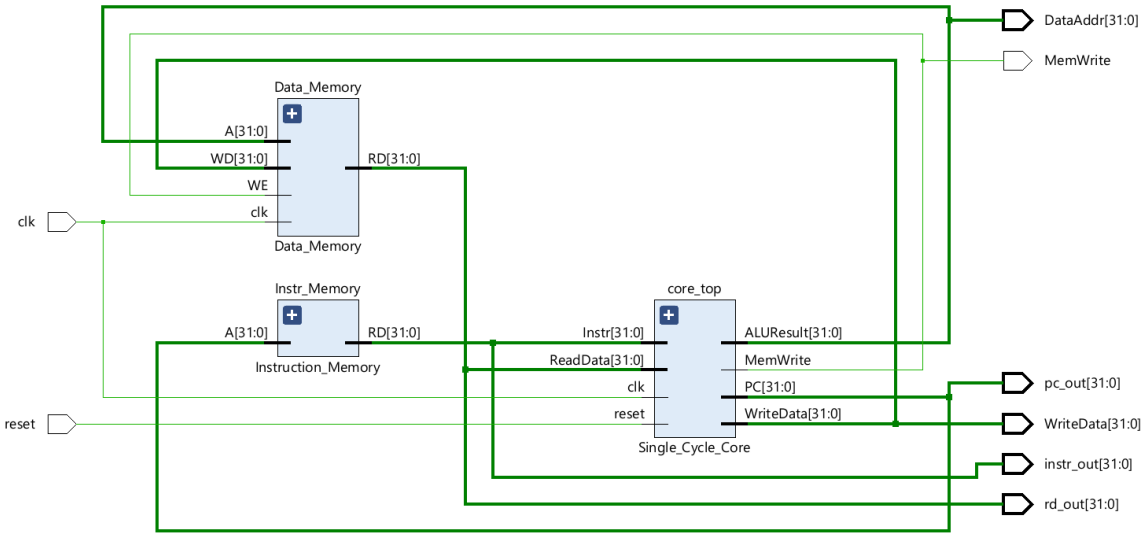


Figure 3.2.2: RTL Netlist generated by Vivado EDA

3.2.1 Datapath Architecture

The datapath forms the computational backbone of the processor and controls the movement of data between different hardware modules during instruction execution. The SMVDU-AHO-32 processor utilizes a single-cycle datapath architecture in which every instruction completes execution within one clock cycle.

The datapath operation consists of the following stages:

1. Instruction Fetch
2. Instruction Decode
3. Register Read
4. Arithmetic/Logical Execution
5. Memory Access
6. Write Back

During instruction fetch, the Program Counter generates the address of the instruction to be fetched from instruction memory. The fetched instruction is decoded by the Control Unit and ALU Decoder, which generate the necessary control signals for datapath operation.

Source operands are read from the Register File and supplied to the Arithmetic Logic Unit (ALU). Depending on the instruction type, the ALU performs arithmetic operations, logical

operations, memory address calculation, or branch target computation. For load and store instructions, the Data Memory module is accessed using the ALU-generated memory address.

Finally, the computed result or loaded memory data is written back into the destination register through the write-back network.

The single-cycle datapath architecture simplifies control logic implementation and enables easier hardware verification and FPGA prototyping.

3.2.2 Program Counter (PC)

The Program Counter (PC) is a 32-bit sequential register responsible for storing the address of the next instruction to be executed. During normal instruction execution, the Program Counter increments by 4 because each RV32I instruction occupies 32 bits (4 bytes).

For branch and jump instructions, the Program Counter loads the computed target address generated by the branch logic circuitry. The PC update operation is controlled through multiplexing logic driven by branch and jump control signals.

The Program Counter was implemented using resettable D flip-flops with an active-high asynchronous reset signal. During reset conditions, the PC initializes to the boot address 0x00000000.

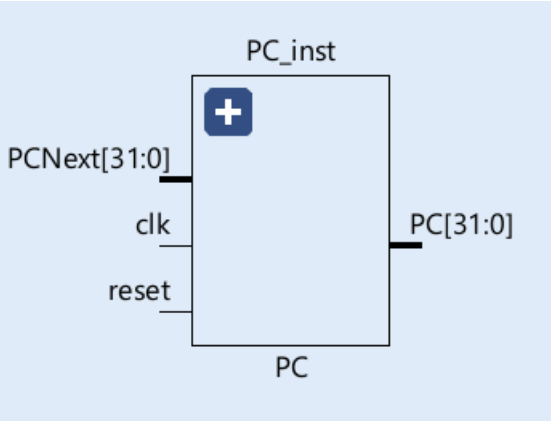


Figure 1.2.2.1: Program Counter

3.2.3 Instruction Memory

The Instruction Memory module stores the machine-code instructions executed by the processor. The memory was implemented as a word-aligned read-only memory structure using synthesizable Verilog constructs.

The instruction fetch address is generated by the Program Counter and supplied to the instruction memory. Since RV32I instructions are 32 bits wide and word aligned, the lower two address bits are ignored during memory access.

Instruction fetching is performed using:

Instruction_Address[31:2]

This simplifies address decoding and aligns instruction access with the 32-bit datapath organization.

The instruction memory contains manually encoded RV32I machine instructions used for simulation and FPGA verification.

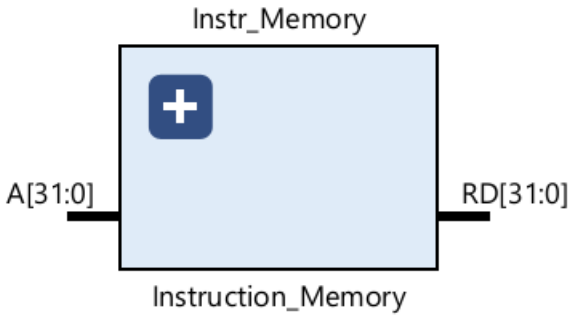


Figure 3.2.3.1: Instruction Memory

3.2.4 Register File

The Register File is one of the most important components of the processor datapath. It contains thirty-two 32-bit General Purpose Registers (GPRs) used for storing operands, intermediate values, and execution results.

The register file architecture includes:

- Two asynchronous read ports
- One synchronous write port
- 32-bit data width
- 5-bit register addressing

The read ports allow simultaneous access to two source operands during instruction execution, while the write port updates the destination register at the positive edge of the clock.

In compliance with the RISC-V specification, register x0 is permanently hardwired to logic zero, and write operations targeting this register are ignored.

The register file was implemented using synthesizable Verilog array structures and synchronous sequential logic.

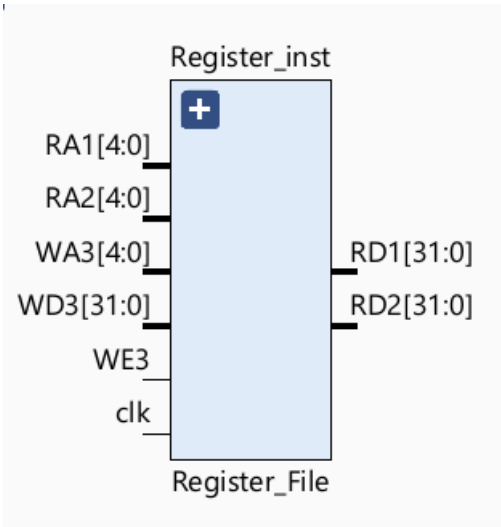


Figure 3.2.4.1: Register File: 32-bit

3.2.5 Immediate Generation Unit

The Immediate Generation Unit extracts immediate fields from instructions and performs sign extension according to the instruction format.

The unit supports:

- I-Type immediates
- S-Type immediates
- B-Type immediates
- U-Type immediates
- J-Type immediates

The extracted immediates are extended to 32 bits before being supplied to the ALU or branch address generation logic.

The Immediate Generation Unit was implemented using combinational bit-slicing and concatenation operations in Verilog HDL.

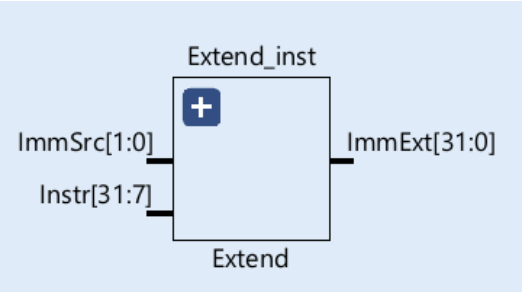


Figure 3.2.5.1: Immediate Extender

3.2.6 Arithmetic Logic Unit (ALU)

The Arithmetic Logic Unit (ALU) performs all arithmetic and logical operations within the processor datapath. The ALU receives operands from the Register File or Immediate Generation Unit and generates the corresponding execution result.

The ALU supports:

- Addition
- Subtraction
- AND
- OR
- XOR
- Shift operations
- Set Less Than (SLT)

The operation performed by the ALU is controlled using a 3-bit ALUControl signal generated by the ALU Decoder.

To optimize hardware resource utilization, subtraction operations were implemented using two's complement addition over the existing adder circuitry instead of dedicated subtraction hardware.

The ALU also generates status outputs including the Zero flag used for branch condition evaluation.

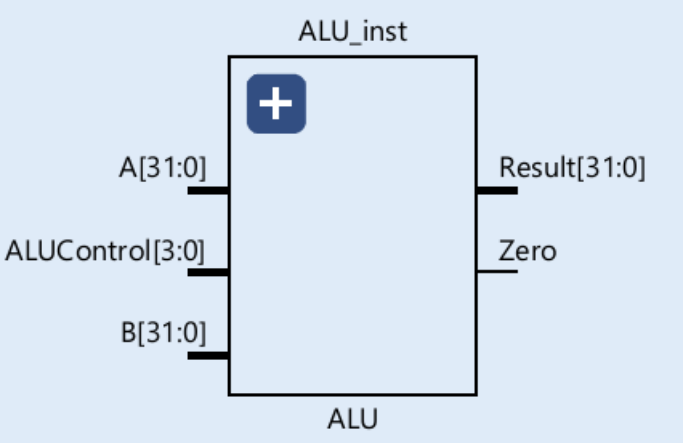


Figure 3.2.6.1: Arithmetic & Logic Unit

3.2.7 Control Unit and ALU Decoder

The Control Unit is responsible for decoding instructions and generating datapath control signals required for processor execution.

The Control Unit consists of:

- Main Decoder
- ALU Decoder

The Main Decoder interprets the opcode field and generates high-level control signals such as:

- RegWrite
- ALUSrc
- MemWrite
- ResultSrc
- Branch
- Jump

The ALU Decoder further interprets the funct3 and funct7 fields to generate the required ALUControl signal for arithmetic and logical operations.

The complete control logic was implemented using combinational Verilog logic to minimize control latency and simplify datapath timing.

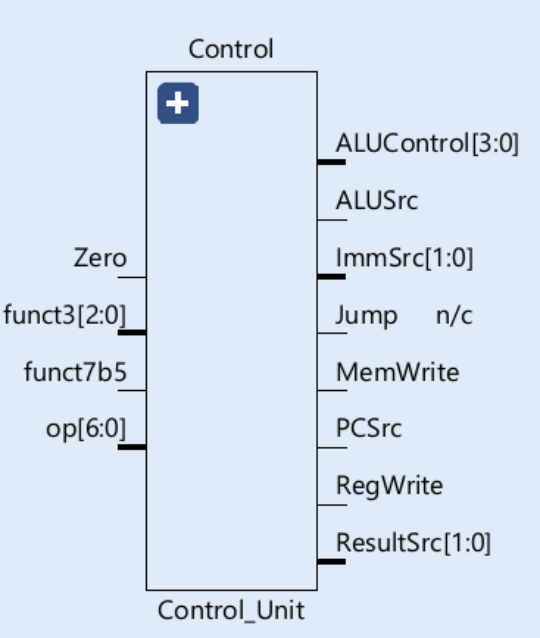


Figure 3.2.7.1: Control Unit

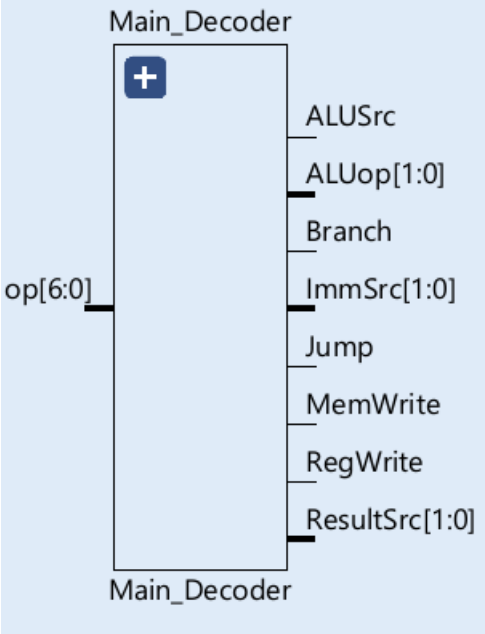


Figure 3.2.7.2: Main Decoder

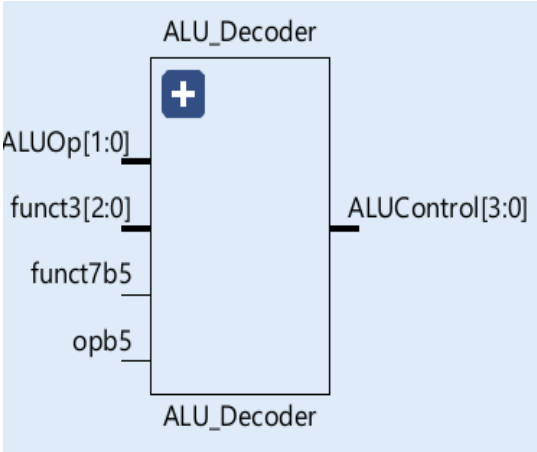


Figure 3.2.7.3: ALU Decoder

3.2.8 Data Memory

The Data Memory module stores runtime data accessed during load and store instructions. The memory supports read and write operations controlled through the MemWrite control signal.

The memory interface operates using word-aligned addressing and is accessed using addresses generated by the ALU during memory instructions.

Write operations occur synchronously with the clock signal, while read operations are implemented combinationally for simplified single-cycle execution.

During FPGA implementation, the memory structures were inferred into FPGA Block RAM (BRAM) resources by the synthesis tool.

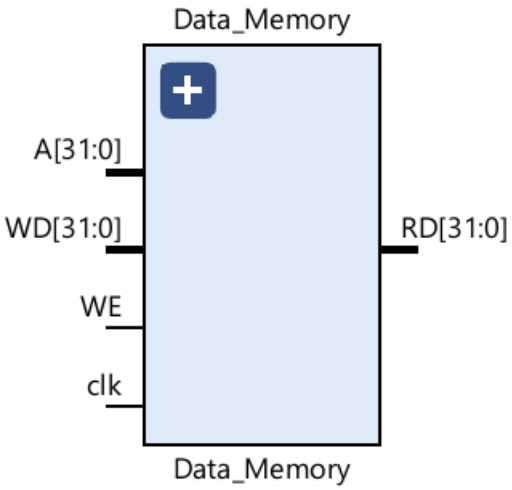


Figure 3.2.8.1: Data Memory

3.2.9 Branch and Jump Logic

The branch and jump logic circuitry controls Program Counter redirection during conditional and unconditional control transfer operations.

Branch target addresses are generated using:

- Current Program Counter value
- Sign-extended immediate offset

Conditional branch decisions are evaluated using ALU comparison outputs and zero-flag logic. Depending on the branch condition result, the Program Counter either increments sequentially or loads the computed target address.

Jump instructions directly update the Program Counter with the computed jump target while simultaneously storing the return address into the destination register.

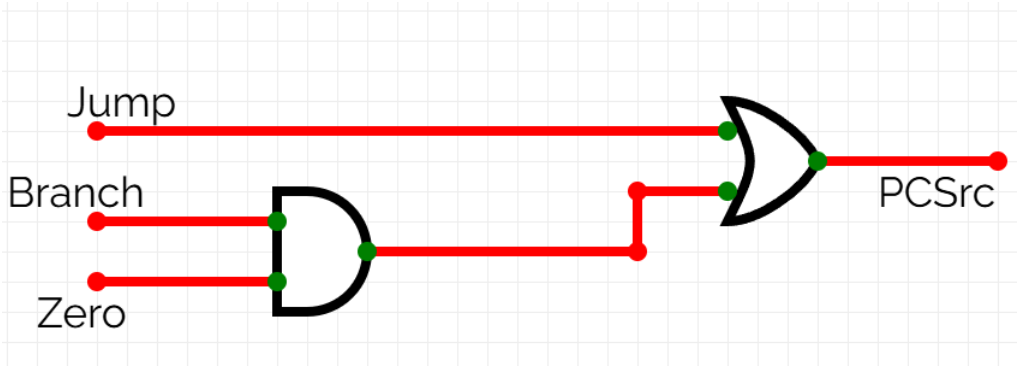


Figure 3.2.9.1: Brach and Jump Logic

3.2.10 Multiplexer Network

Multiple multiplexers are utilized throughout the datapath to control operand selection, memory access routing, and write-back operations.

Major multiplexing operations include:

- ALU operand selection
- Program Counter source selection
- Write-back data selection
- Branch target selection

The multiplexers are controlled using signals generated by the Control Unit and branch logic circuitry.

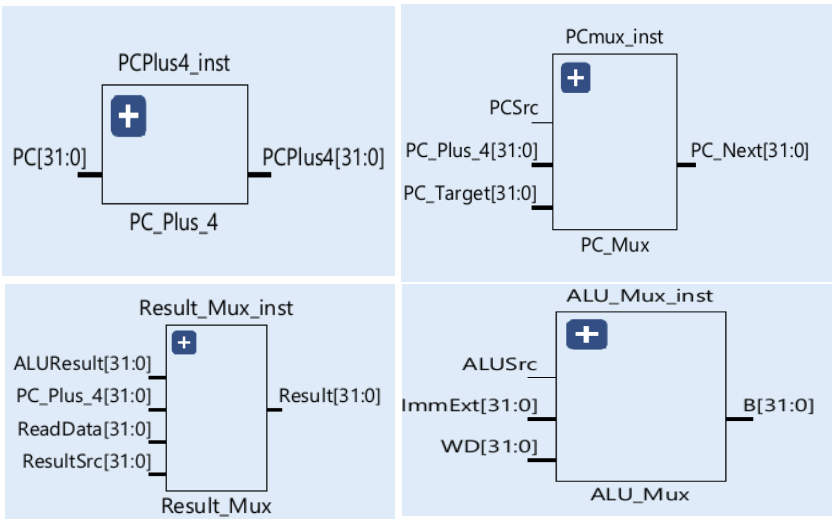


Figure 2: Muxes in DataPath

3.2.11 Clock and Reset Architecture

The SMVDU-AHO-32 processor operates using a single synchronous clock domain. All sequential elements including the Program Counter and Register File are triggered on the positive edge of the clock signal.

The architecture utilizes an active-high asynchronous reset signal to initialize the processor during startup conditions. The reset logic initializes the Program Counter and clears sequential storage elements to ensure deterministic processor operation.

During FPGA implementation on the PYNQ-Z2 platform, the system clock was generated using onboard clock management resources and constrained through Xilinx Design Constraints (XDC) files for timing analysis and implementation.

3.3 Verification Strategy

Prior to FPGA prototyping and ASIC implementation, the SMVDU-AHO-32 processor was developed using strictly synthesizable IEEE 1364-2005 Verilog HDL. The RTL design methodology emphasized modularity, synthesizability, portability, and compatibility across both FPGA and Cadence ASIC implementation flows. To ensure reliable synthesis and predictable hardware generation, the RTL design strictly utilized standard synthesizable constructs while avoiding unsupported behavioural descriptions.

Special care was taken to eliminate unintended hardware inference during synthesis. Internal tri-state buffers, inferred latches, incomplete sensitivity lists, and non-synthesizable procedural constructs were systematically avoided to ensure deterministic hardware behavior and compatibility with standard-cell synthesis methodologies.

The processor architecture was hierarchically partitioned into multiple functional modules to simplify debugging, verification, and synthesis optimization. The major RTL partitions included:

1. **Core Top Module:**
   The top-level integration module responsible for interconnecting the datapath, control logic, instruction memory, and data memory interfaces.
2. **Instruction Decode Unit:**

A combinational decoding block responsible for extracting opcode, rs1, rs2, rd, funct3, and funct7 fields from the fetched instruction and generating the corresponding control signals.

3. **Immediate Generation Unit (ImmGen):**
   A combinational logic block responsible for extracting and sign-extending immediate values for I-Type, S-Type, B-Type, U-Type, and J-Type instructions using bit concatenation and replication logic.
4. **Arithmetic Logic Unit and Branch Control:**
   The ALU performs arithmetic and logical operations while simultaneously generating status outputs such as the Zero flag used for branch evaluation and conditional Program Counter updates.
5. **Register File:**
   A 32 × 32-bit register array with dual asynchronous read ports and a synchronous write port for operand access and result storage.
6. **Memory Interface Modules:**
   Separate instruction and data memory modules implementing Harvard architecture organization for independent instruction fetch and data access operations.

Functional correctness of the processor was verified through extensive RTL simulation using custom Verilog testbenches and standard RISC-V assembly instruction sequences. The verification process validated:

- arithmetic operations,
- logical operations,
- immediate instruction execution,
- memory load/store functionality,
- branch conditions,
- jump operations,
- and register write-back behavior.

Waveform analysis was performed to monitor critical internal signals including:

- Program Counter updates,
- ALU outputs,
- register file contents,
- control signals,
- branch decisions,
- and memory transactions.

The successful completion of RTL verification ensured that the processor architecture operated correctly before proceeding to FPGA synthesis and ASIC implementation stages.

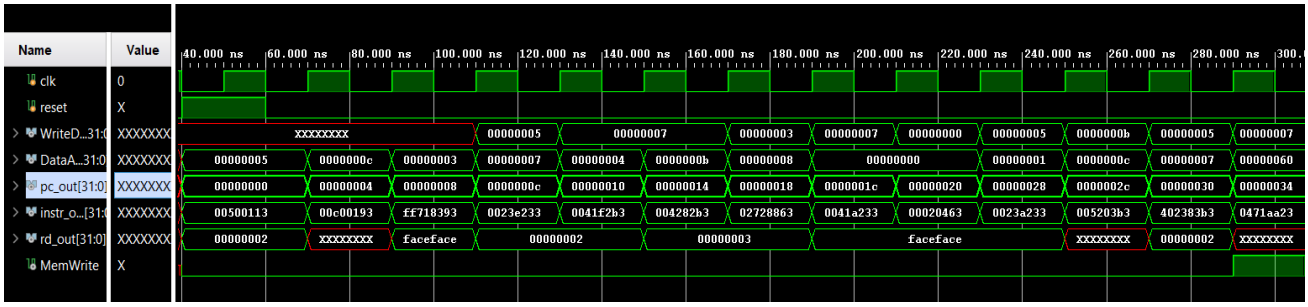


Figure 3.3.1: Simulation Waveform obtained from testbench

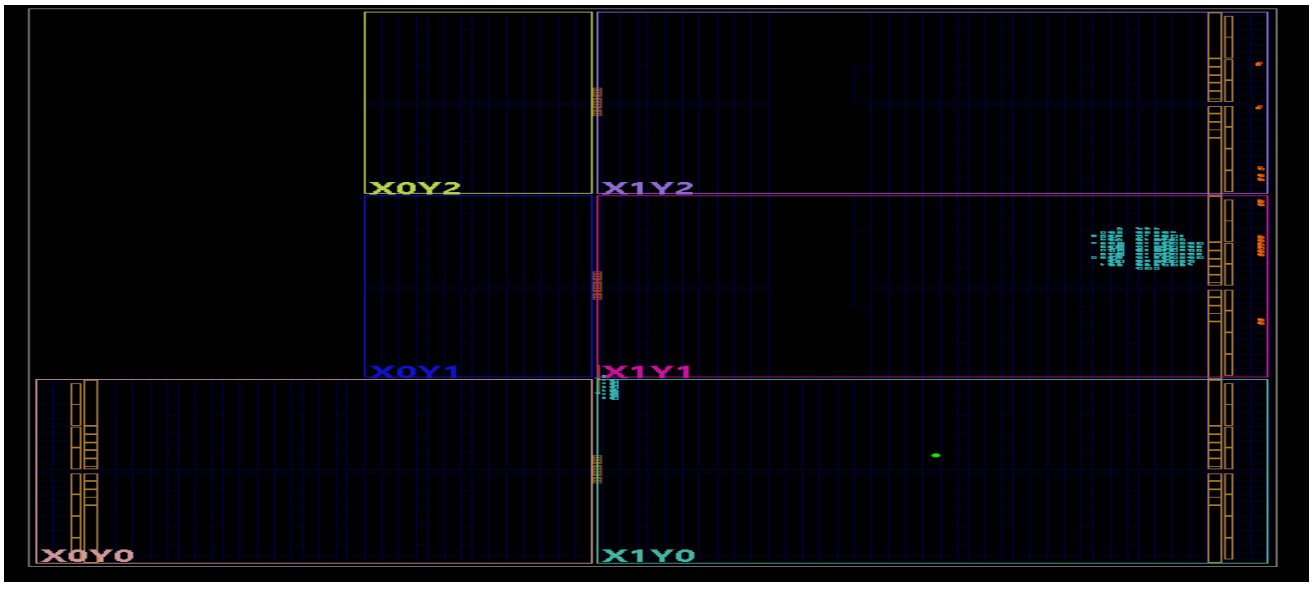


Figure 3.3.2: Post Simulation Layout

3.4 FPGA Prototyping and Hardware Verification

After successful RTL simulation and functional verification, the SMVDU-AHO-32 processor was implemented on the PYNQ-Z2 for real-time hardware validation. FPGA prototyping served as an intermediate verification stage prior to ASIC implementation, enabling validation of processor functionality, timing behavior, hardware resource utilization, and clock-domain operation under realistic execution conditions.

The FPGA implementation flow included:

- RTL synthesis
- constraint definition
- placement and routing
- bitstream generation
- Design Rule Check (DRC)
- timing analysis
- power analysis
- and on-board hardware testing.

The complete FPGA implementation was carried out using Vivado Design Suite.

3.4.1 FPGA Architecture

The FPGA implementation platform used in this project was based on the Xilinx Zynq-7000 architecture integrated within the PYNQ-Z2 development board.

The Zynq-7000 architecture combines:

- Programmable Logic (PL)
- and Processing System (PS)

within a single System-on-Chip (SoC) device.

The Programmable Logic section contains configurable logic resources including:

- Look-Up Tables (LUTs)
- Flip-Flops (FFs)

- Block RAMs (BRAMs)
- DSP slices
- Clock Management Tiles (CMTs)
- and programmable routing interconnects.

The FPGA fabric enables implementation of custom digital systems such as processors, accelerators, and communication controllers using Hardware Description Languages (HDLs).

The SMVDU-AHO-32 processor was implemented entirely within the Programmable Logic (PL) section of the FPGA fabric.

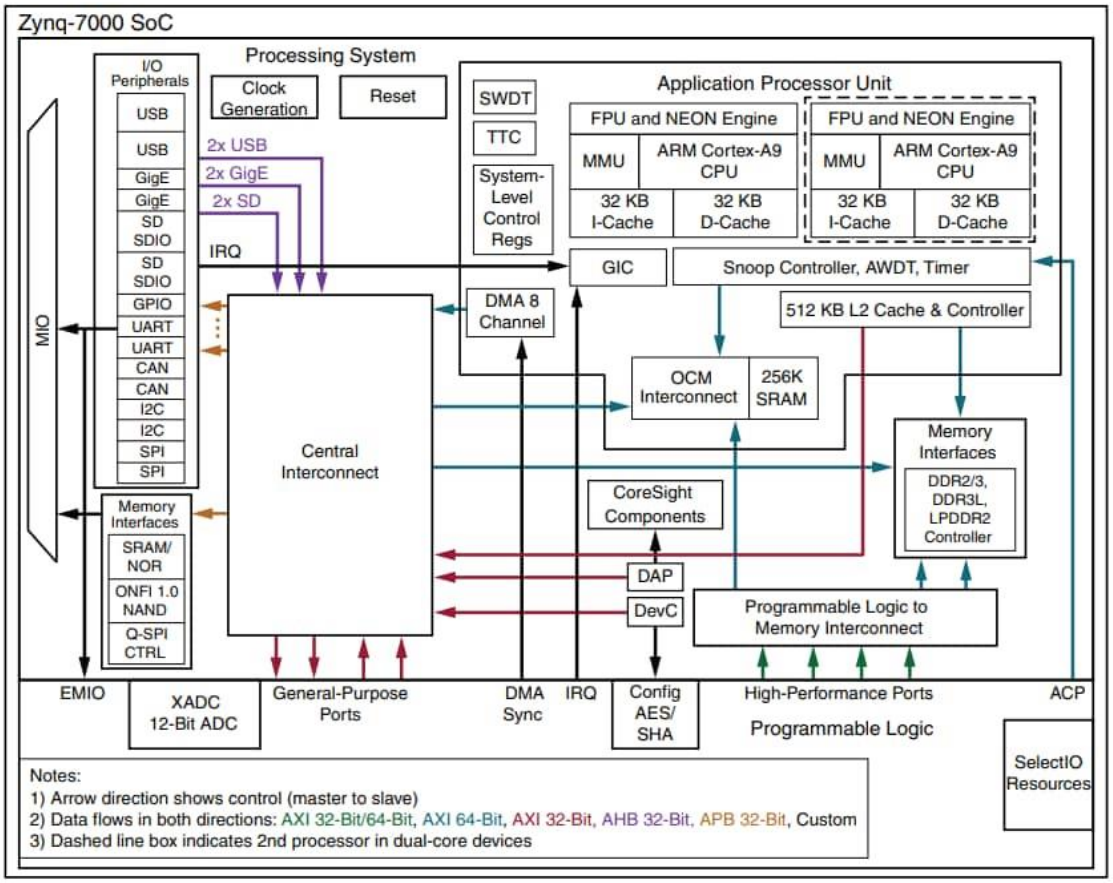


Figure 3.4.1.1: Top-level architecture of PYNQ-Z2(Zynq-7000 SoC)

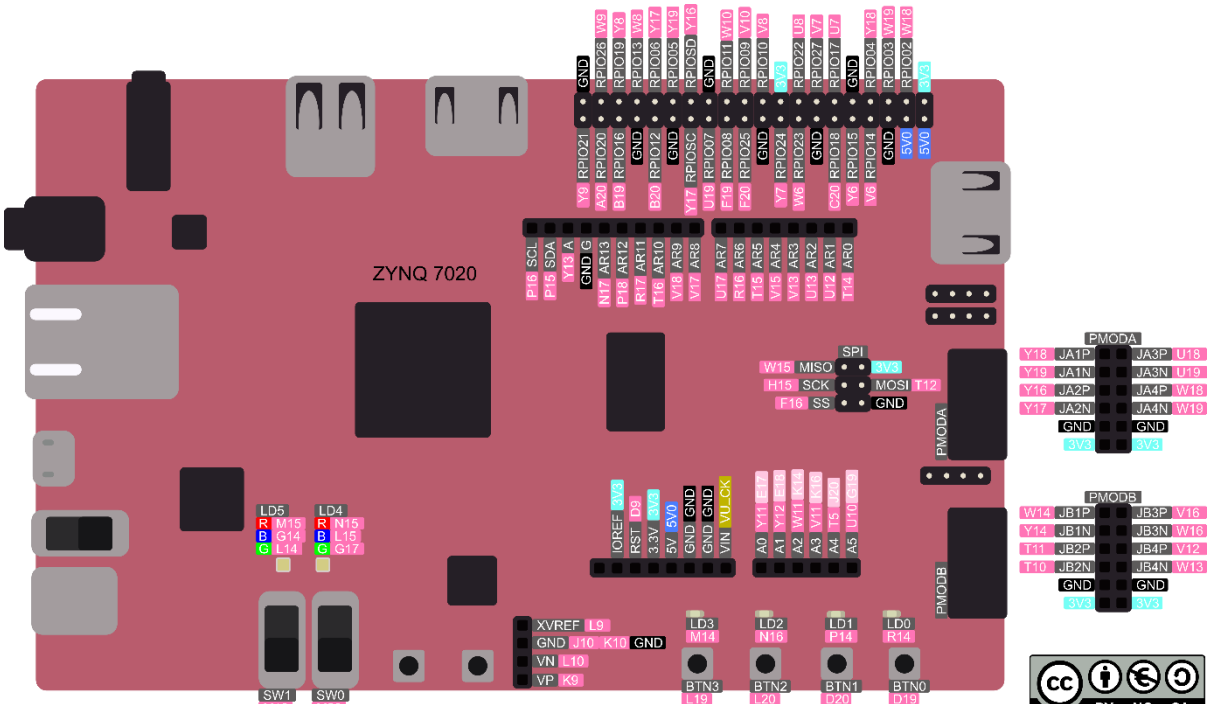


Figure 3.4.1.2: Pinout of Pynq-Z2

3.4.2 PYNQ-Z2 Development Board

The PYNQ-Z2 is an FPGA development platform based on the Xilinx Zynq-7000 xc7z020-1clg400c device. The board provides a balanced combination of programmable logic resources, onboard peripherals, clocking infrastructure, and memory interfaces suitable for embedded system prototyping and processor implementation.

Table 3.4.2.1: Major features of PYNQ-Z2

Feature	**Specification**
FPGA Device	**Xilinx Zynq-7000 xc7z020**
Logic Cells	**~85K**
Block RAM	**4.9 Mb**
DSP Slices	**220**
Onboard Clock	**125 MHz**
DDR Memory	**512 MB DDR3**
GPIO Interfaces	**LEDs, Switches, Buttons**
Communication Interfaces	**UART, HDMI, Ethernet, USB**

The onboard 125 MHz clock source was utilized as the primary system clock for processor implementation and verification.

3.4.3 FPGA Pin Configuration and Constraint File

FPGA pin assignments and timing constraints were specified using Xilinx Design Constraint (XDC) files. The XDC file defines:

- physical FPGA pin mapping,
- I/O standards,
- clock constraints,
- and timing specifications required for synthesis and implementation.

The primary clock input was mapped using:

```
set_property -dict { PACKAGE_PIN H16 IOSTANDARD LVCMOS33 } [get_ports { clk }]

create_clock -add -name sys_clk_pin -period 8.00 -waveform {0 4} [get_ports { clk }]
```

The PACKAGE_PIN constraint specifies the physical FPGA package pin connected to the onboard oscillator, while the IOSTANDARD defines the electrical signaling standard used by the FPGA I/O interface.

The create_clock constraint defines the operating clock period for timing analysis and synthesis optimization. An 8 ns clock period corresponds to a 125 MHz operating frequency.

Additional constraints were used for:

- reset input mapping,
- GPIO assignments,
- LED interfaces,
- and debugging signals.

Proper constraint specification was essential for achieving successful timing closure and stable FPGA operation.

3.4.4 FPGA Synthesis and Implementation

The SMVDU-AHO-32 RTL design was synthesized using the Vivado synthesis engine targeting the xc7z020 FPGA device. During synthesis, the RTL description was translated into FPGA primitives including LUTs, Flip-Flops, carry-chain logic, and Block RAM resources.

The implementation flow included:

1. RTL Elaboration
2. Logic Synthesis
3. Technology Mapping
4. Placement
5. Routing
6. Timing Optimization
7. Bitstream Generation

The processor datapath was implemented primarily using LUT-based combinational logic and sequential Flip-Flop resources. Memory structures were inferred into FPGA Block RAM resources wherever applicable.

3.4.5 FPGA Functional Verification

After synthesis and implementation, FPGA-based hardware verification was performed to validate processor functionality under real operating conditions.

The FPGA verification process included:

- Program Counter monitoring
- Instruction execution validation
- Register write-back verification
- Memory read/write testing
- Branch and jump operation testing
- Clock and reset validation

Custom RV32I instruction sequences were executed to verify arithmetic, logical, branch, and memory operations.

Waveform debugging and signal monitoring were performed using:

- simulation waveforms,
- internal signal observation,
- and FPGA debugging methodologies.

The FPGA implementation successfully executed the programmed instruction sequences without functional errors.

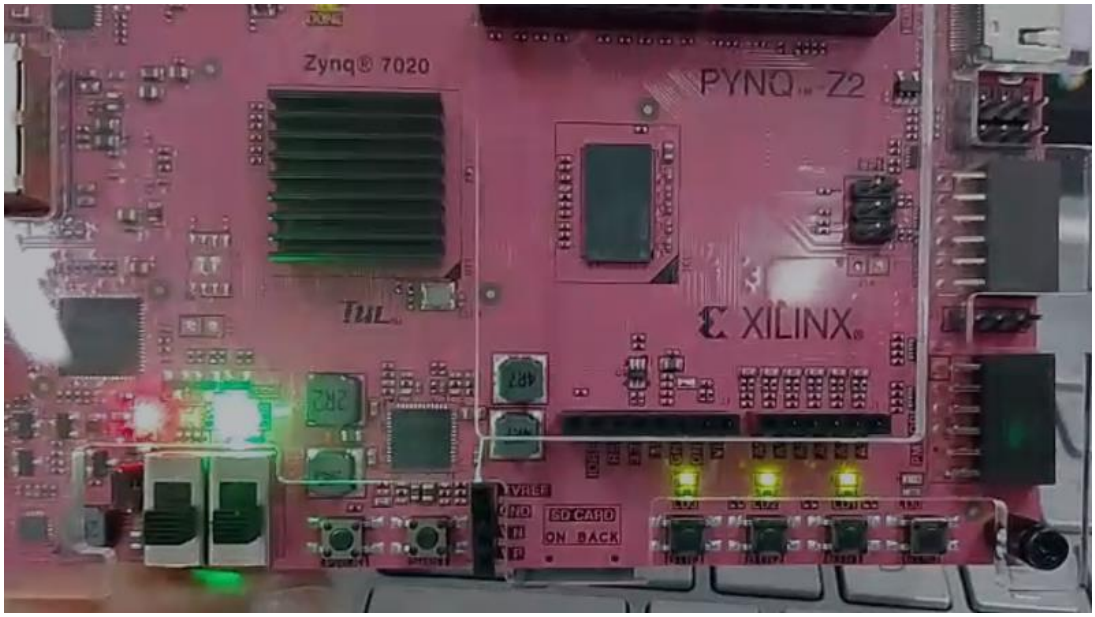


Figure 3.4.5.1: Program Counter is Configured into these led

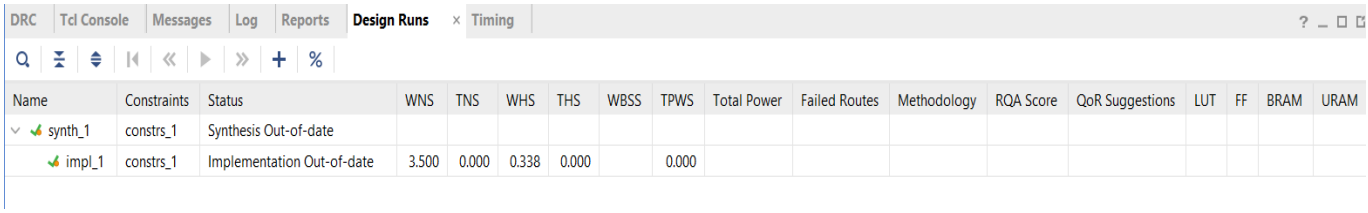
3.4.6 Design Rule Check (DRC)

After routing completion, Design Rule Check (DRC) analysis was performed using Vivado implementation tools to verify that the generated FPGA design satisfied all architectural and electrical implementation constraints.

The DRC process validated:

- routing legality,
- clocking constraints,
- I/O configuration correctness,
- timing rule compliance,
- and resource utilization consistency.

The final implementation completed DRC verification successfully without critical design violations, confirming that the FPGA design was implementation-safe and hardware compatible.



Name	Constraints	Status	WNS	TNS	WHS	THS	WBSS	TPWS	Total Power	Failed Routes	Methodology	RQA Score	QoR Suggestions	LUT	FF	BRAM	URAM
synth_1	constrs_1	Synthesis Out-of-date															
impl_1	constrs_1	Implementation Out-of-date	3.500	0.000	0.338	0.000		0.000									

Figure 3.4.6.1: DRC Report for Design

3.4.7 FPGA Timing and Power Analysis

Static timing analysis was performed after routing to evaluate setup timing, hold timing, clock uncertainty, and critical path delay.

The timing analysis confirmed successful timing closure at the target operating frequency. Critical timing paths were primarily associated with:

- ALU computation,
- memory access,
- and write-back datapath routing.

Power analysis was also performed using Vivado Power Analysis tools. The analysis included:

- dynamic switching power,
- clock network power,
- logic power,
- signal power,
- and static leakage power.

The processor demonstrated relatively low power consumption due to:

- single-cycle architecture simplicity,
- lightweight datapath organization,
- and absence of complex pipeline or cache structures.

The FPGA implementation results validated the functional correctness and hardware feasibility of the SMVDU-AHO-32 processor before proceeding to ASIC implementation stages.

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power:	**0.104 W**
Design Power Budget:	**Not Specified**
Process:	typical
Power Budget Margin:	**N/A**
Junction Temperature:	**26.2°C**
Thermal Margin:	58.8°C (4.9 W)
Ambient Temperature:	25.0 °C
Effective ϑJA:	11.5°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	Low

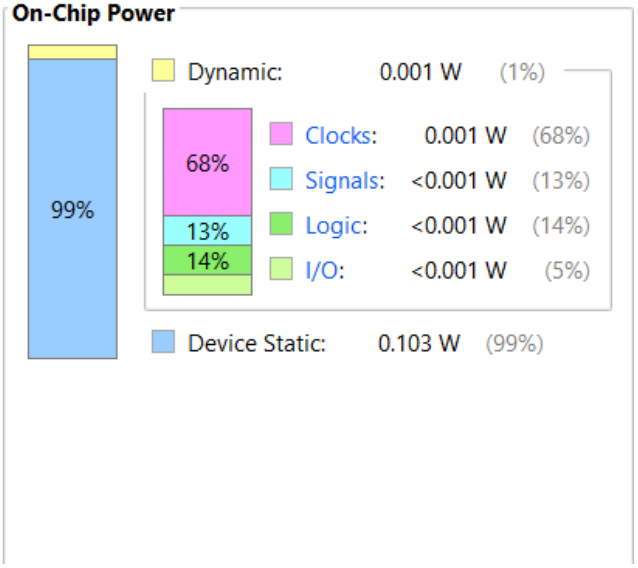


Figure 3.4.7.1: On Chip Power report

CHAPTER 4 - PHASE 2 (PART I): ASIC FRONTEND FLOW

4.1 Functional Verification (Cadence IRun)

The standard-cell ASIC frontend transforms the technology-independent Verilog representation into a technology-mapped gate-level netlist. Before synthesis, the golden RTL underwent rigorous simulation using the Cadence IRun simulation tool. A self-checking testbench (`Exhaustive_TB.v`) was developed, instantiating the core (`Core_Datapath.v` and `Instruction_Memory.v`) and feeding an assembly-compiled HEX file containing extensive corner-case operations.

The simulation elaborated the design hierarchy successfully without functional errors, generating native compiled code for execution. The `irun` utility simulated the operation for 100,000 ps (100 ns), exhaustively verifying all ISA instruction types.

Listing 4.1: Cadence IRun Simulation Log Summary

```
ncsim> run
=================================================
 Starting Exhaustive Verification (Backup Core)
=================================================
[100000 ns] MEMORY WRITE: Addr = 0x00000064 | Data = 0x000000ff
=================================================
 [SUCCESS] All ISA Types Tested Successfully!
 Hardware is verified and ready for Synthesis.
=================================================
Simulation stopped via $stop(1) at time 100 NS + 0
```

The generated signals were visualized and verified using Cadence SimVision, confirming the correct state transitions, memory writes, and program counter increments across the execution timeline.

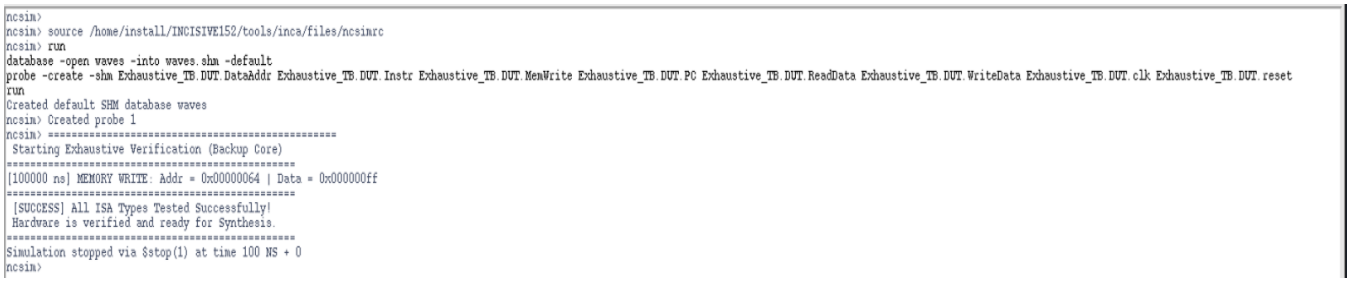


Figure 4.1.1: Cadence SimVision Console Execution Log

Figure 4.1.2: Cadence SimVision Waveform showing functional execution

4.2 Code Coverage Analysis

To evaluate the robustness of the verification environment, code coverage was extracted using Cadence IMC (Integrated Metrics Center). We started the code coverage analysis with the testbench `tb_Single_Cycle_Top.v` (detailed in Appendix F.1).

The Testbench Implementation Since our memory is already populated with the instruction set test sequences, the testbench is extremely straightforward. It just needs to provide the clock and reset, and monitor the simulation.

The simulation, with coverage tracking enabled, was executed using the following `irun` command:

```
irun -coverage all -covoverwrite ../TESTBENCHES/tb_Single_Cycle_Top.v
../Golden_Design_RTL/*.v
```

Coverage Results Summary: The generated coverage database was loaded into Cadence IMC for analysis. Based on the test program execution, the tool automatically detected the hit and missed logic points. The baseline coverage results for the initial execution are as follows:

- **Overall Average Grade:** 61.44%
- **DUT (Design Under Test):** 61.67%
- **Control Unit:** 73.18%
- **Datapath:** 54.39%
- **Instruction Memory:** 75.00%

- **Data Memory:** 67.86%

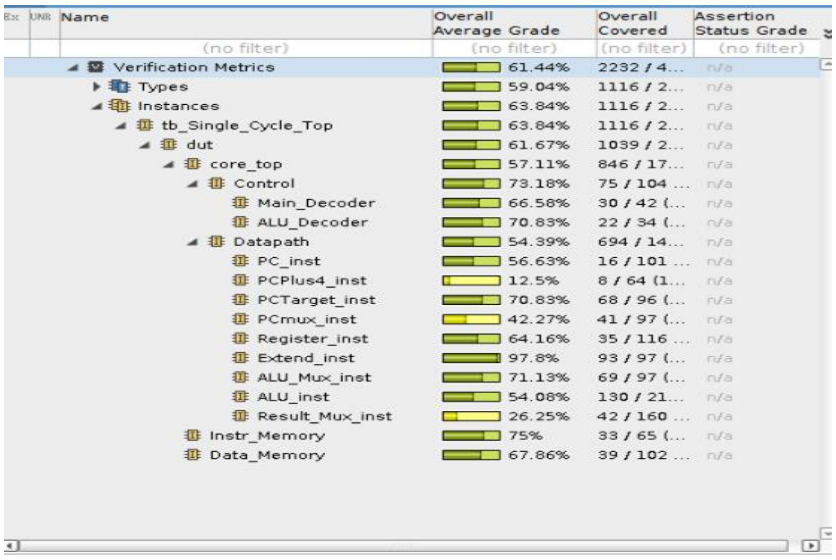


Figure 4.2.1: Cadence IMC Coverage Analysis - Overall Metrics

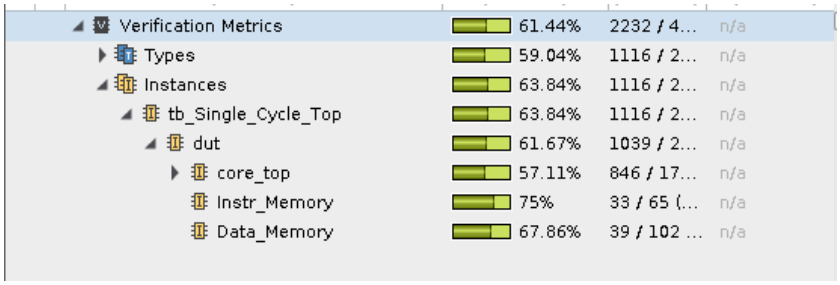


Figure 4.2.2: Cadence IMC Coverage Analysis - Datapath Hierarchy

4.3 SRAM Macro Generation (OpenRAM)

Standard ASIC logic synthesis tools cannot efficiently map large memory arrays to standard logic gates (Flip-Flops) without incurring catastrophic area and routing congestion penalties. Therefore, the internal memory arrays were not written as standard synthesizable RTL for the ASIC flow. Instead, **OpenRAM**, an open-source memory compiler, was utilized to generate dedicated hard-macros.

Based on the updated architectural partitioning, a specialized single-port SRAM macro named `sram_32x64_180nm` was compiled targeting a 180nm process node.

Macro Specifications:

- **Word Size (Data Width):** 32 bits

- **Words (RAM Depth):** 64 words
- **Address Width:** 6 bits
- **Physical Dimensions:** 918.2 µm x 599.8 µm
- **Control Signals:** `clk0` (Clock), `csb0` (Active-low Chip Select), `web0` (Active-low Write Enable).

The compiler successfully produced the required views for the physical design flow:

1. **Behavioral Verilog (`sram_32x64_180nm.v`):** A simulation model including read/write logic and customizable hold delays (`T_HOLD = 1`), used during Gate-Level Simulation.
2. **Library Exchange Format (`sram_32x64_180nm.lef`):** Contains the abstract layout representation, including the bounding box dimensions (`SIZE 918.2 BY 599.8`) and physical metal layer pin definitions (e.g., `LAYER metal4` for `din0` pins) required by the Cadence Innovus router.
3. **GDSII Layout (`sram_32x64_180nm.gds`):** The final geometric mask data containing the actual transistor-level routing of the bitcells and decoders.

These files were treated as physical hard-macros during the backend flow, demanding strict pin alignment and ring routing.

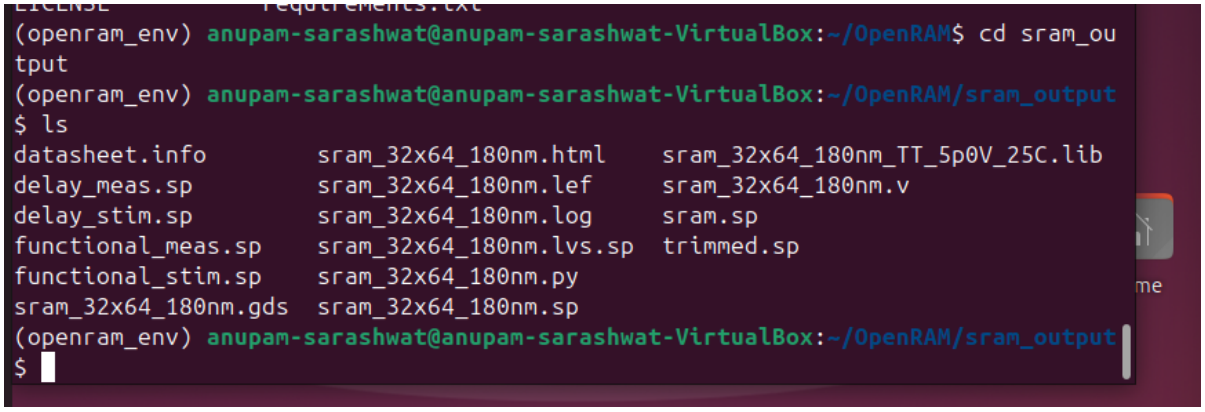


Figure 4.3.1: OpenRAM Generated SRAM Macro (sram_32x64_180nm) Layout and Files

4.4 Synthesis & Timing Constraints (Iterative Analysis)

Logic synthesis translates abstract behavior into physical standard cells targeting a predictive 180nm standard cell library alongside the OpenRAM `.lib` timing models. This process was executed using the Cadence Genus Synthesis Solution, driven by strict Synopsys Design Constraints (SDC) establishing a **12.0 ns** clock period (83.3 MHz target).

Synthesis Run 1: The Flattening Problem The initial logic synthesis attempt resulted in a completely flattened, scattered netlist where standard cells were not bifurcated into their respective modules. Furthermore, the synthesis tool's aggressive optimization algorithms pruned critical logic, falsely treating memory boundaries as unconnected constant values. This resulted in a heavily optimized but structurally inaccurate design showing an artificially low cell area (~142,705 μm^2) and an incomplete power profile (~12.5 mW).

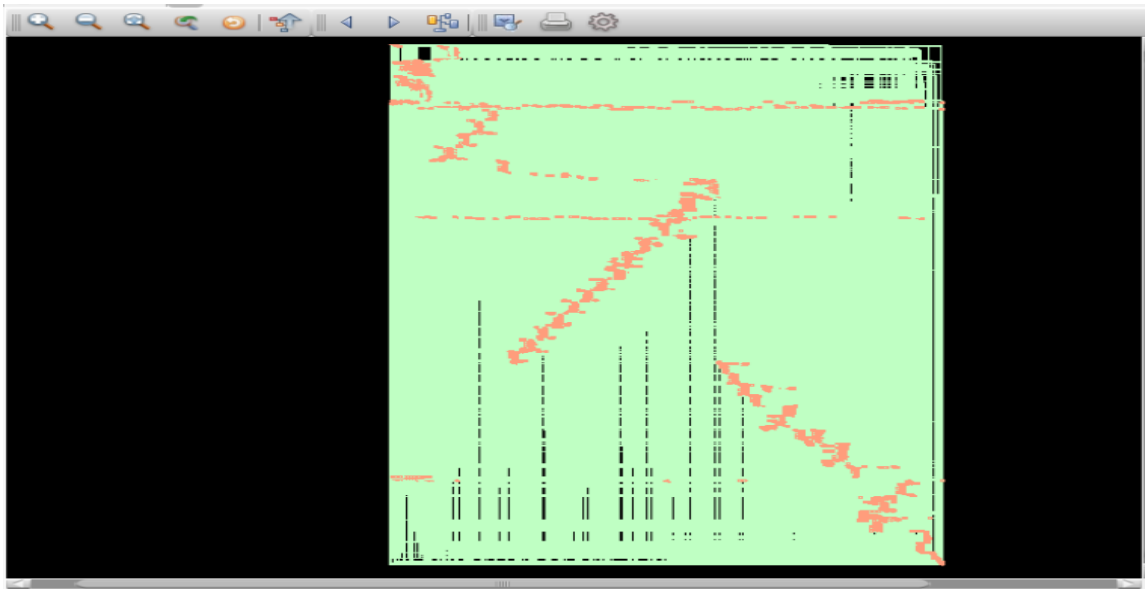


Figure 4.4.1: Synthesis Run 1

Synthesis Run 2: Hierarchy Preservation & Full System Solution To resolve the aggressive pruning and lack of partitioning, the system was encapsulated within a new custom SoC Wrapper (`Single_Cycle_Top.v`). This wrapper was explicitly designed to expose critical `PC` and `Instr` buses to the module boundary, preventing the synthesizer from classifying internal logic as dead code.

Crucially, the Cadence Genus TCL script (detailed in Appendix C.1) was significantly upgraded. Root-level boundary protection and anti-pruning commands were introduced prior to the `syn_generic` phase to enforce module-by-module partitioning:

```
set_db / .boundary_opto false
set_db [get_db modules Single_Cycle_Top] .preserve_hierarchy true
set_db / .auto_ungroup none
set_db / .optimize_constant_0_flops false
set_db / .delete_unloaded_insts false
```

The second synthesis iteration successfully preserved the module hierarchy, strictly bifurcating the logic into `Data_Memory`, `Instruction_Memory`, and `Single_Cycle_Core`, completely resolving the flattening issue.

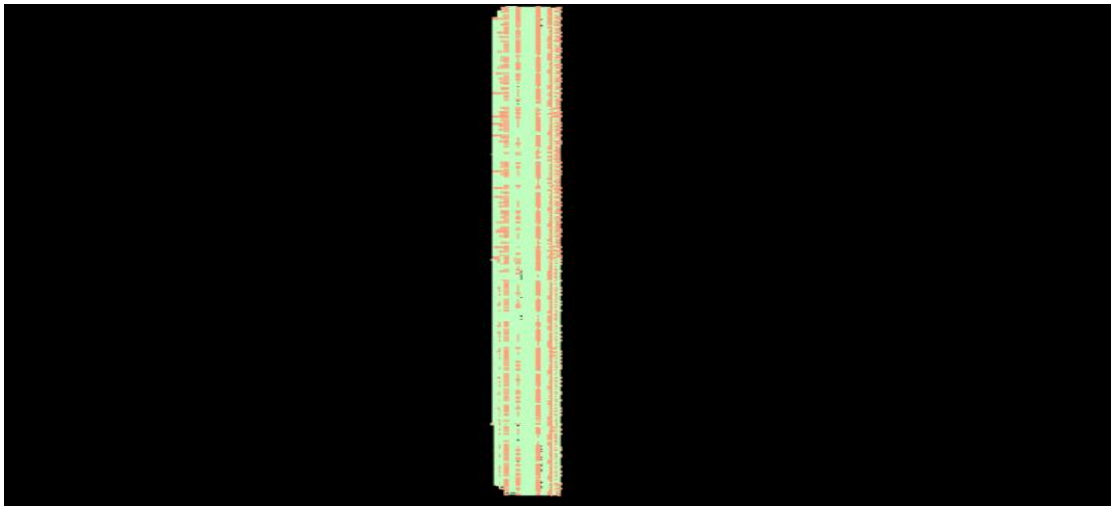


Figure 4.4.2: Cadence Genus Synthesis GUI - Instruction Memory Hierarchy

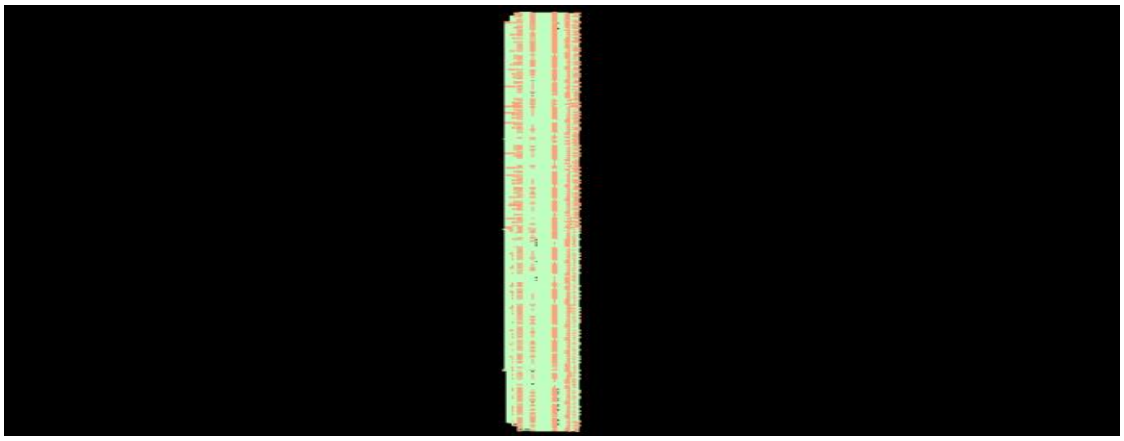


Figure 4.4.3: Cadence Genus Synthesis GUI - Data Memory Hierarchy

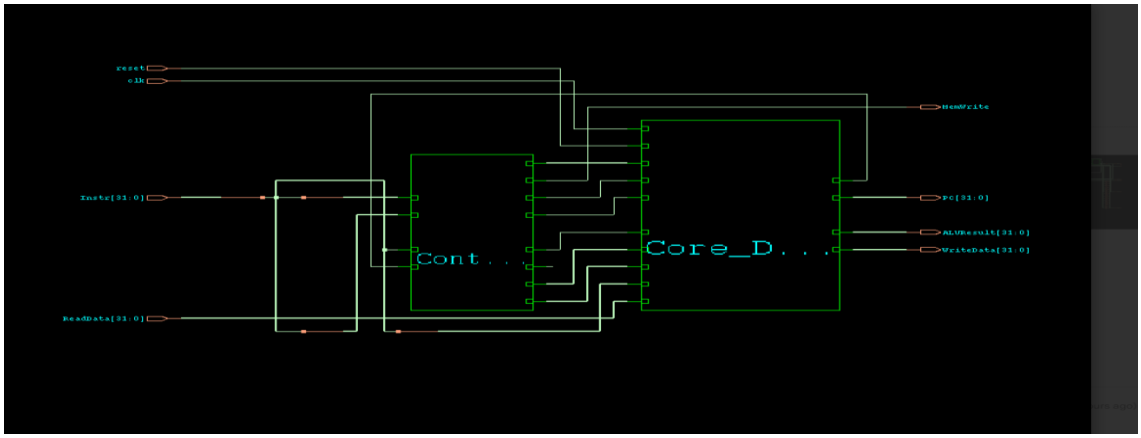


Figure 4.4.4: Cadence Genus Synthesis GUI - Core Block Hierarchy

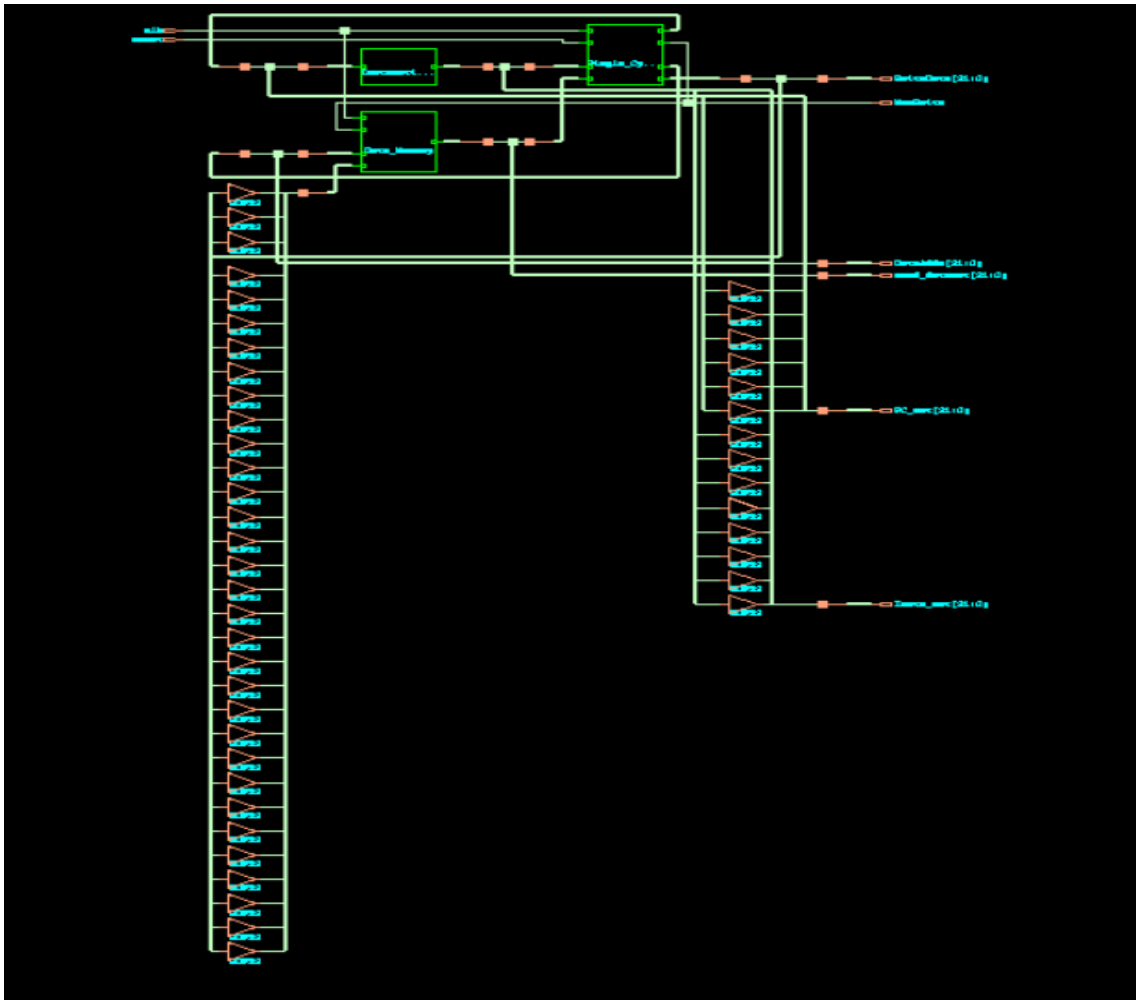


Figure 4.4.5: Cadence Genus Synthesis GUI - Top SoC Module Bifurcation

Comparative Analysis (Run 1 vs Run 2): The successful preservation of the memory modules and standard cell hierarchy yielded the accurate metrics for the full system implementation:

Table 4.4.1: Comparative Analysis

Metric	**Run 1 (Flattened/Pruned Logic)**	**Run 2 (Full System Preserved)**
Total Cell Area	~142,705 μm^2	350,343.102 μm^2
Standard Cell Count	N/A (Flattened)	12,343 Instances
Total Power Consumption	~12.5 mW	18.06 mW (1.80686×10^{-2} W)

- **Area Distribution:** The full system occupies 350,343.102 μm^2 This is explicitly bifurcated into `Data_Memory` (180,550.340 μm^2, `Instruction_Memory` (32,462.338 μm^2) and the internal `core_top` logic (136,718.367 μm^2).
- **Power Delta:** Total system power is 18.06 mW. This jump accurately models the new memory arrays and the forced retention of logic states during operation compared to the falsely optimized 12.5 mW.
- **Timing Closure:** The full system achieved a positive setup slack of **+58 ps**. The longest data path propagation required exactly 7,442 ps against a required time of 7,500 ps, safely meeting the 12,000 ps (12.0 ns) timing constraint.

4.5 Gate Level Simulation (GLS) & LEC

Following the successful synthesis and structural preservation of the core modules, the generated standard-cell mapped netlist (`full_system_netlist.v`) was subjected to **Gate Level Simulation (GLS)**. To rigorously validate the functional integrity of the synthesized logic against real-world gate delays, the exact same verification testbench (`tb_Single_Cycle_Top.v`) was utilized to drive the netlist.

Side-by-Side Verification Analysis: To guarantee absolute behavioral fidelity, the Golden RTL design was simulated alongside the synthesized netlist simultaneously. The output waveforms from both the baseline RTL (Golden Design) and the synthesized netlist (GLS) were loaded concurrently into Cadence SimVision for a side-by-side comparative analysis.

Key architectural signals—including the Program Counter transitions (`PC_out`), fetched Instructions (`Instr_out`), computed ALU Results/Memory Addresses (`DataAddr`), and Memory Write flags (`MemWrite`)—were monitored and matched cycle-by-cycle. The results perfectly aligned without cycle mismatches, explicitly confirming that standard-cell technology mapping and propagation delays did not introduce functional glitches or break the instruction execution pipeline.

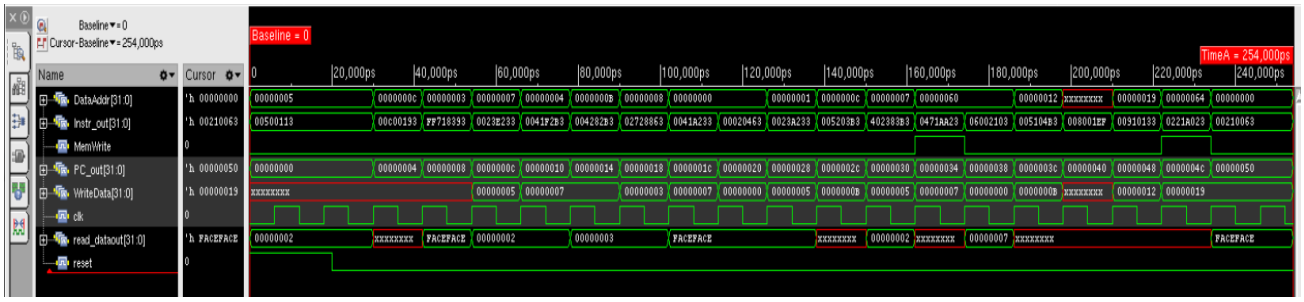


Figure 4.5.1: SimVision GLS vs Golden RTL Output Verification

```
Created default SHM database waves
xcelium> Created probe 1
xcelium> ================================================
   SMVDU-AH0-32 Processor Simulation Started
================================================
Time: 24000  | PC: 00000000 | Instr: 00500113 | MemWrite: 0 | Addr: 00000005 | WData: xxxxxxxx
Time: 36000  | PC: 00000004 | Instr: 00c00193 | MemWrite: 0 | Addr: 0000000c | WData: xxxxxxxx
Time: 48000  | PC: 00000008 | Instr: ff718393 | MemWrite: 0 | Addr: 00000003 | WData: xxxxxxxx
Time: 60000  | PC: 0000000c | Instr: 0023e233 | MemWrite: 0 | Addr: 00000007 | WData: 00000005
Time: 72000  | PC: 00000010 | Instr: 0041f2b3 | MemWrite: 0 | Addr: 00000004 | WData: 00000007
Time: 84000  | PC: 00000014 | Instr: 004282b3 | MemWrite: 0 | Addr: 0000000b | WData: 00000007
Time: 96000  | PC: 00000018 | Instr: 02728863 | MemWrite: 0 | Addr: 00000008 | WData: 00000003
Time: 108000 | PC: 0000001c | Instr: 0041a233 | MemWrite: 0 | Addr: 00000000 | WData: 00000007
Time: 120000 | PC: 00000020 | Instr: 00020463 | MemWrite: 0 | Addr: 00000000 | WData: 00000000
Time: 132000 | PC: 00000028 | Instr: 0023a233 | MemWrite: 0 | Addr: 00000001 | WData: 00000005
Time: 144000 | PC: 0000002c | Instr: 005203b3 | MemWrite: 0 | Addr: 0000000c | WData: 0000000b
Time: 156000 | PC: 00000030 | Instr: 402383b3 | MemWrite: 0 | Addr: 00000007 | WData: 00000005
Time: 168000 | PC: 00000034 | Instr: 0471aa23 | MemWrite: 1 | Addr: 00000060 | WData: 00000007
Time: 180000 | PC: 00000038 | Instr: 06002103 | MemWrite: 0 | Addr: 00000060 | WData: 00000000
Time: 192000 | PC: 0000003c | Instr: 005104b3 | MemWrite: 0 | Addr: 00000012 | WData: 0000000b
Time: 204000 | PC: 00000040 | Instr: 008001ef | MemWrite: 0 | Addr: xxxxxxxx | WData: xxxxxxxx
Time: 216000 | PC: 00000048 | Instr: 00910133 | MemWrite: 0 | Addr: 00000019 | WData: 00000012
Time: 228000 | PC: 0000004c | Instr: 0221a023 | MemWrite: 1 | Addr: 00000064 | WData: 00000019
Time: 240000 | PC: 00000050 | Instr: 00210063 | MemWrite: 0 | Addr: 00000000 | WData: 00000019
Time: 252000 | PC: 00000050 | Instr: 00210063 | MemWrite: 0 | Addr: 00000000 | WData: 00000019
================================================
   Test Program Completed Successfully.
================================================
Simulation complete via $finish(1) at time 254 NS + 0
./tb_Single_Cycle_Top.v:60        $finish;
xcelium>
```

Figure 4.5.2: SimVision GLS vs Golden RTL Output Verification

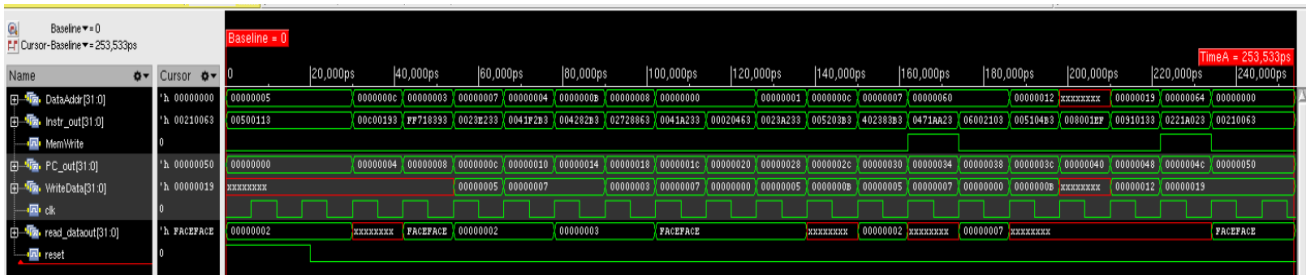


Figure 4.5.2: SimVision GLS vs Golden RTL Output Verification 3

```
   SMVDU-AH0-32 Processor Simulation Started
================================================
Time: 24000  | PC: 00000000 | Instr: 00500113 | MemWrite: 0 | Addr: 00000005 | WData: xxxxxxxx
Time: 36000  | PC: 00000004 | Instr: 00c00193 | MemWrite: 0 | Addr: 0000000c | WData: xxxxxxxx
Time: 48000  | PC: 00000008 | Instr: ff718393 | MemWrite: 0 | Addr: 00000003 | WData: xxxxxxxx
Time: 60000  | PC: 0000000c | Instr: 0023e233 | MemWrite: 0 | Addr: 00000007 | WData: 00000005
Time: 72000  | PC: 00000010 | Instr: 0041f2b3 | MemWrite: 0 | Addr: 00000004 | WData: 00000007
Time: 84000  | PC: 00000014 | Instr: 004282b3 | MemWrite: 0 | Addr: 0000000b | WData: 00000007
Time: 96000  | PC: 00000018 | Instr: 02728863 | MemWrite: 0 | Addr: 00000008 | WData: 00000003
Time: 108000 | PC: 0000001c | Instr: 0041a233 | MemWrite: 0 | Addr: 00000000 | WData: 00000007
Time: 120000 | PC: 00000020 | Instr: 00020463 | MemWrite: 0 | Addr: 00000000 | WData: 00000000
Time: 132000 | PC: 00000028 | Instr: 0023a233 | MemWrite: 0 | Addr: 00000001 | WData: 00000005
Time: 144000 | PC: 0000002c | Instr: 005203b3 | MemWrite: 0 | Addr: 0000000c | WData: 0000000b
Time: 156000 | PC: 00000030 | Instr: 402383b3 | MemWrite: 0 | Addr: 00000007 | WData: 00000005
Time: 168000 | PC: 00000034 | Instr: 0471aa23 | MemWrite: 1 | Addr: 00000060 | WData: 00000007
Time: 180000 | PC: 00000038 | Instr: 06002103 | MemWrite: 0 | Addr: 00000060 | WData: 00000000
Time: 192000 | PC: 0000003c | Instr: 005104b3 | MemWrite: 0 | Addr: 00000012 | WData: 0000000b
Time: 204000 | PC: 00000040 | Instr: 008001ef | MemWrite: 0 | Addr: xxxxxxxx | WData: xxxxxxxx
Time: 216000 | PC: 00000048 | Instr: 00910133 | MemWrite: 0 | Addr: 00000019 | WData: 00000012
Time: 228000 | PC: 0000004c | Instr: 0221a023 | MemWrite: 1 | Addr: 00000064 | WData: 00000019
Time: 240000 | PC: 00000050 | Instr: 00210063 | MemWrite: 0 | Addr: 00000000 | WData: 00000019
Time: 252000 | PC: 00000050 | Instr: 00210063 | MemWrite: 0 | Addr: 00000000 | WData: 00000019
================================================
   Test Program Completed Successfully.
================================================
Simulation complete via $finish(1) at time 254 NS + 0
./tb_Single_Cycle_Top.v:60        $finish;
xcelium>
```

Figure 4.5.3: SimVision GLS vs Golden RTL Output Verification 4

Logical Equivalence Checking (LEC): While GLS verifies dynamic functionality over time, optimization algorithms can inadvertently alter abstract logic relationships. Cadence Conformal **Logical Equivalence Check (LEC)** was utilized via a `.do` script to formally verify that the unoptimized Golden RTL design and the post-synthesis gate-level netlist were mathematically equivalent across all possible input permutations.

Listing 4.2: Cadence Conformal LEC Log Summary

```
================================================================================
=====
Compared points      PO     DFF       Total
--------------------------------------------------------------------------------
-----
Equivalent           97     1024      1121
================================================================================
=====
```

The formal mathematical verification proved that the synthesized gate-level netlist is **100% logically equivalent** to the original RTL. Cadence Conformal successfully mapped all 97 Primary Outputs (POs) and all 1,024 active Flip-Flops (DFFs), proving they behave identically under every condition.

Handling Unmapped Key Points (Register x0 Optimization): During the LEC run, a specific warning was encountered: `// Warning: Golden has 32 unmapped key points which will not be compared`. This behavior is completely expected and architecturally correct. In the RISC-V ISA, Register `x0` is strictly hardwired to zero. During synthesis, Cadence Genus recognized that the 32 flip-flops for Register 0 never change state. To aggressively save silicon area and leakage power, Genus pruned these flip-flops and tied their outputs directly to ground (Logic 0).

Because these 32 flip-flops exist in the behavioral RTL array but were physically deleted in the netlist, Conformal flagged them as "Unmapped". The tool successfully mapped the remaining 1,024 active flip-flops (Registers `x1` through `x31`) and confirmed their perfect equivalence, successfully green-lighting the netlist for the physical backend flow.

4.6 Design for Testability (DFT) and Scan Chain Insertion

4.6.1 Why Do We Need DFT?

When foundries like TSMC fabricate the `Single_Cycle_Core`, they are literally printing billions of microscopic transistors using light (lithography) onto a silicon wafer. This process

is incredibly precise, but it is not perfect. Dust particles, microscopic variations in silicon doping, or tiny imperfections in the light masks can cause manufacturing defects:

- **Stuck-at Faults:** A physical wire is accidentally shorted to Power (Stuck-at-1) or Ground (Stuck-at-0).
- **Bridging Faults:** Two adjacent microscopic wires accidentally touch each other.
- **Transition Faults:** A gate is slightly malformed, so it takes too long to transition from 0 to 1, causing a setup violation at the next flip-flop.

While Gate Level Simulation (GLS) and Logical Equivalence Checking (LEC) prove the mathematical logic works perfectly, they only simulate perfect gates. When the physical chip comes back from the factory, a tiny speck of dust could ruin a single NAND gate deep inside the ALU. It is computationally prohibitive to run the entire 21-instruction test program on millions of fabricated chips to check them, and functional testing rarely isolates the exact broken gate.

4.6.2 The Solution: Scan Chains

DFT solves this by modifying the physical netlist *after* synthesis. Originally, the 1,024 registers (flip-flops) in the design are connected normally: data goes into `D`, and out of `Q`. During DFT insertion, the synthesis tool (Cadence Genus) removes every normal `DFF` and replaces it with a **Scan Flip-Flop (SDFF)**.

A Scan Flip-Flop has a built-in multiplexer at the input, controlled by a Shift Enable (`SE`) signal:

- **Normal Mode (SE = 0):** It acts like a regular flip-flop (`D` to `Q`).
- **Scan Mode (SE = 1):** It ignores the normal logic and instead takes input from a special Scan In (`SI`) pin.

The DFT tool wires all 1,024 flip-flops together in a massive daisy-chain (a giant shift register) from a special `scan_in` pin on the outside of the chip, all the way to a `scan_out` pin.

Factory Testing Procedure:

1. Put the chip in Scan Mode (`SE = 1`).
2. Shift 1,024 bits of specific test data directly into every single flip-flop on the chip via `scan_in`.

3. Switch back to Normal Mode (`SE = 0`) for *exactly one clock cycle*. The physical logic gates process the test data.
4. Switch back to Scan Mode (`SE = 1`).
5. Shift all 1,024 bits *out* of the chip via `scan_out` and compare them to the expected results generated by an ATPG (Automatic Test Pattern Generation) tool.

This methodology allows Automatic Test Equipment (ATE) to test every single internal gate on the chip in milliseconds.

4.6.3 Tcl Scripts, Constraints, and Tool Execution

To implement this, specific DFT constraints and instructions were added to the Cadence Genus Tcl script (`run_dft_core.tcl`):

Pre-Synthesis DFT Setup:

```
set_db / .dft_scan_style muxed_scan
define_shift_enable -name SE -active high -create_port SE -default
set_db / .use_scan_seqs_for_non_dft false
```

Post-Mapping DFT Insertion:

```
set_db [get_db designs *] .dft_min_number_of_scan_chains 1
define_scan_chain -name top_chain -sdi scan_in -sdo scan_out -create_ports
check_dft_rules
convert_to_scan
connect_scan_chains -auto_create_chains
```

Upon running these commands, Cadence Genus performed the physical logic replacement. The synthesis log (`genus.log9`) confirms the 100% successful mapping of all active registers:

```
Scan mapping: converting flip-flops that pass TDRC.
Scan mapping done: 1024 flip-flops mapped to scan.
Category                          Number   Percentage
------------------------------------------------------------
Scan flip-flops mapped for DFT      1024       100.00%
```

Every single active flip-flop inside the processor was replaced with a TSMC SDFF. Subsequently, Genus successfully linked these registers into a unified chain:

```
Starting DFT Scan Configuration for module 'Single_Cycle_Core'
Configuring 1 chains for 1024 scan f/f
top_chain (scan_in -> scan_out) has 1024 registers; Domain:clk, edge: rising
```

4.6.4 Secondary Logical Equivalence Check (LEC II)

To guarantee that the scan chain insertion did not break the processor's functional ability to execute RISC-V assembly code, a secondary Logical Equivalence Check (LEC II) was performed. This compared the pre-DFT netlist (Golden) against the post-DFT netlist (Revised).

The check was executed via Cadence Conformal using the command: `lec -dofile dft_lec.do -nogui`

The verification was driven by the following `.do` script:

```
// 1. Setup log file
set log file dft_lec.log -replace

// 2. Read the REAL definitions from the .lib files first
read          library          /home/student/ANUPAM/Fresh_GPDK180/slow.lib
../Memory_Macros/sram_32x64_180nm_TT_5p0V_25C.lib -liberty -both

// 3. APPEND the Verilog simulation models and patches
read   library   /home/student/ANUPAM/Fresh_GPDK180/slow_vdd1v0_basicCells.v
../GLS/patch.v -verilog -both -append

// 4. Read PRE-DFT Netlist as Golden reference
read design ../Synthesis/core_netlist.v -verilog -golden
set root module Single_Cycle_Core -golden

// 5. Read POST-DFT Netlist as Revised design
read design ../DFT/core_dft_netlist.v -verilog -revised
set root module Single_Cycle_Core -revised

// 6. CRITICAL DFT CONSTRAINT: Tie Scan Enable to 0
add pin constraint 0 SE -revised

// 7. Switch to verification mode and compare
set system mode lec
add compare point -all
compare

// 8. Report final results
report compare data
```

4.6.5 The Final Verification Summary

By reading the Liberty (`.lib`) files directly, Conformal used the mathematically perfect definitions for complex standard cells. The critical constraint `add pin constraint 0 SE -revised` was applied to ensure the chip was evaluated strictly in its normal functional mode.

The tool reported a perfect result of **100% Equivalence**:

- **Total Compared Points:** 1121
- **Total Equivalent:** 1121
- **Registers (DFF):** All 1024 registers were verified.

- **Outputs (PO):** All 97 primary outputs were verified.

(Note: "Extra" signals mentioned in the Conformal logs for `scan_in` *and* `scan_out` *are expected warnings, as these physical test pins only exist in the Revised DFT netlist and have no functional counterpart in the Golden netlist).*

4.6.6 Conclusion

This mathematical certainty proves that the RISC-V processor's logic is perfectly intact. The design successfully synthesized the RTL into gates, stitched a physical 1,024-bit scan chain, and verified that the scan addition did not break a single instruction or register operation. This completes the **Sign-off for the front-end design**, yielding a netlist that is mathematically solid and fully testable for physical manufacturing.

CHAPTER 5 - PHASE 2 (PART II): ASIC BACKEND & SoC INTEGRATION

5.1 Core Physical Design

The backend Physical Design (PD) flow, executed via Cadence Innovus, translates the logical netlist into physical geometric masks (GDSII). The initial step involved reading the synthesized netlist, the SDC constraints, and the technology LEF containing the physical dimensions of the standard cells.

- **Floorplanning:** The core utilization was targeted at 70% (to leave adequate space for clock tree buffers and routing logic), with an aspect ratio of 1.0 (square die).

Figure 5.1.1: Floorplan

- **Power Planning:** Sub-micron chips are highly susceptible to voltage (IR) drop. A robust Power Distribution Network (PDN) was engineered: Global VDD and VSS rings were synthesized using top-level metal layers (M8, M9), vertical/horizontal straps were dropped, and standard cell rails were routed via the follow-pin methodology.
    1. **Drawn Power rails**

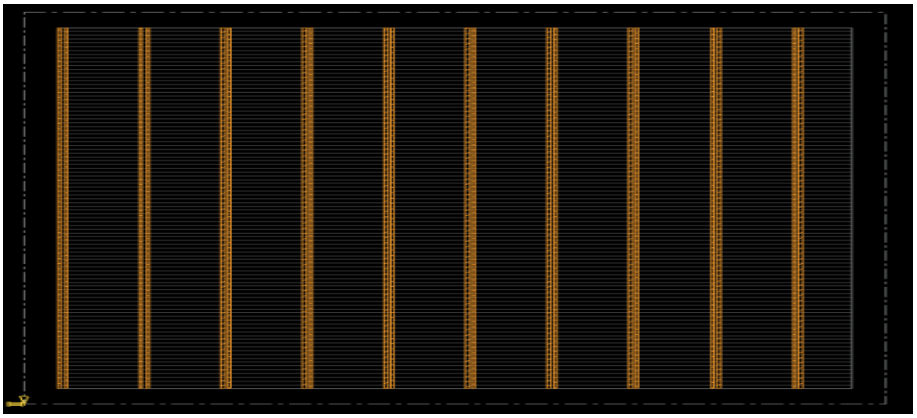


Figure 5.1.2: Drawn Power Rails

2. **Drawn Power Rings**

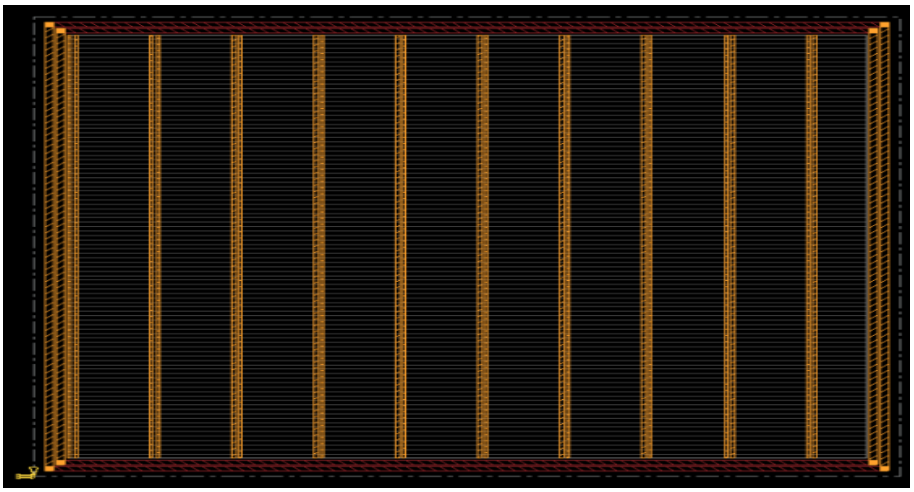


Figure 5.1.3: Drawn Power Rings

3. **Drawn Optimized Power Distribution according to standard cell placement**

- **I/O Pin Placements**

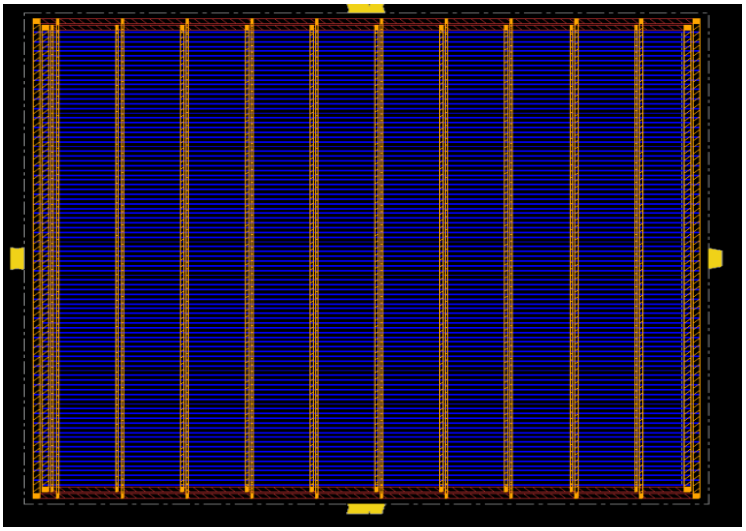


Figure 5.1.4: IO Pin Placement

- **Placement:** Innovus placed the standard cells across the core area. Timing-driven placement grouped logic cells sharing critical timing paths closer together to minimize interconnect propagation delays (t_{pd}).

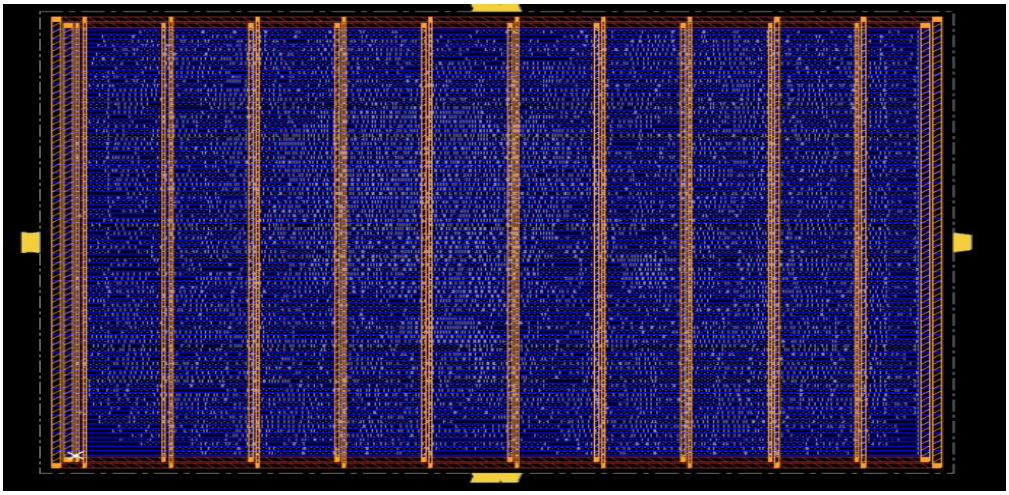


Figure 5.1.5: Placement of Standard Cells

- **Clock Tree Synthesis (CTS):** CTS synthesized a physical H-tree of clock buffers and inverters to route the clock from the primary input pin to every sequential element, minimizing global insertion delay and neutralizing clock skew.

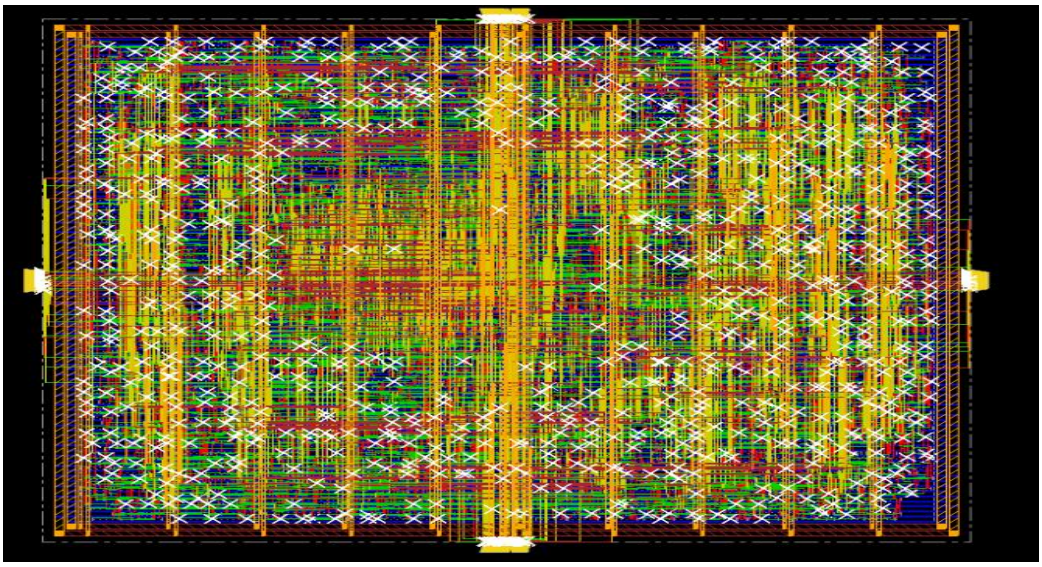


Figure 5.1.6: Clock Tree Synthesis

- **Routing:** NanoRoute physically drew the geometric metal connections and inserted vias between metal layers. Crosstalk and signal integrity rules were strictly enforced.

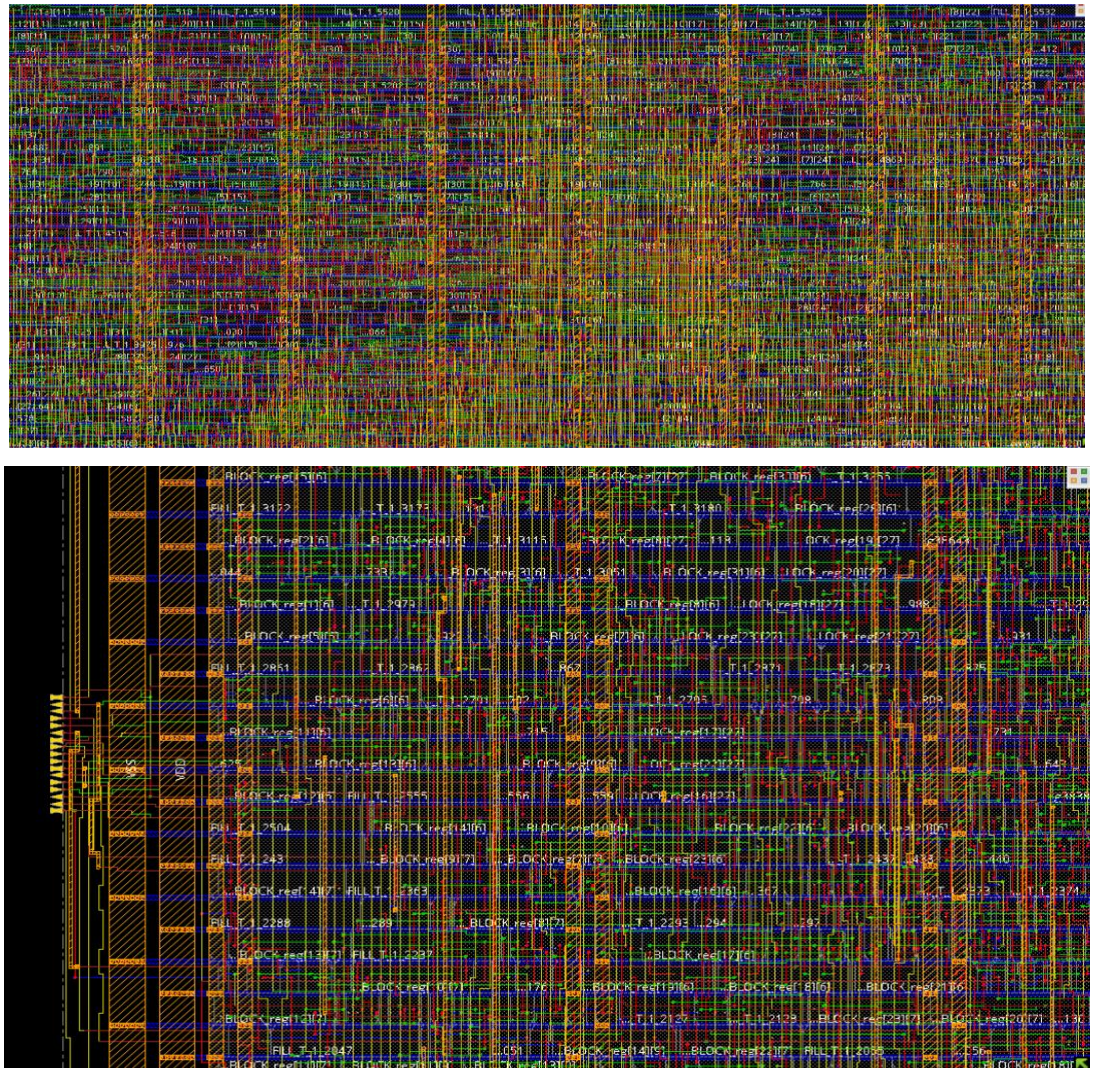


Figure 5.1.7: Post Route View

5.1.1 Final Post-Route ASIC Implementation Summary of core

for SMVDU-AH032 RISC-V Single Cycle Core

Table 5.1.1.1: Final Post Route

Parameter	Final Value
Design Name	`Single_Cycle_Core`
Technology Node	**180nm**
Total Instances	10,708
Total Nets	5,262
Combinational Cells	266
Sequential Cells	178
Tristate Cells	18
Placement Density	~69–71%
RC Extracted Resistors	129,511
Ground Capacitances	134,695

Coupling Capacitances	324,972
Analysis Mode	MMMC + OCV
Signal Integrity Analysis	Enabled
Final WNS	**-0.007 ns**
Final TNS	**-0.012 ns**
Final Violating Paths	**2**
Reg-to-Reg WNS	**+0.632 ns**
Reg-to-Reg Violations	**0**
Routing Status	Successfully Completed
Post-Route Optimization	Completed
DRC Violations	**0**
Total Vias	47,211
Total Routed Wirelength	~400,000 µm
Timing Closure Status	Near Timing Closed

5.1.2 Routing & DRC Analysis

Table 5.1.2.1: Routing & DRC

Parameter	**Observation**
DRC Status	**0 Violations (DRC Clean)**
Routing Layers Used	Metal1 – Metal4
Highest Utilized Layer	Metal3 (~106,594 µm routing)
Routing Quality	Well distributed and congestion-free
Connectivity Issue	1 minor dangling VDD wire stub
Overall Physical Status	Practically Tape-Out Ready

The ASIC backend implementation of the SMVDU-AH032 RISC-V processor core was successfully completed using the RTL2GDS2 design flow in 180nm technology. Placement, clock tree synthesis, routing, parasitic extraction, and post-route timing optimization were performed successfully. The routed design achieved zero DRC violations, proper multilayer routing distribution, and near timing closure with only two minor setup violations. Clean register-to-register timing and successful signal integrity-aware analysis demonstrate that the processor core was physically implemented successfully and is in a near tape-out-ready condition suitable for academic ASIC prototyping.

5.2 SoC Top Wrapper Design & Integration

To realize a complete, self-contained microcontroller, the top-level **SoC Wrapper** (`Single_Cycle_Top.v`) was physically implemented to integrate the core computational logic with its respective memory hierarchy. This macro-integration phase is critical for moving from a standard-cell-only design to a mixed macro/cell System-on-Chip.

The SoC Wrapper integrates:

1. **Core Macro:** The finished physical layout of the processor logic.

2. **Memory Macros:** The Instruction and Data SRAMs generated by OpenRAM.
3. **IO Interface & Pad Ring:** Signal routing between the internal macros and the external chip-level I/O pins, ensuring appropriate buffering for external capacitive loads.

5.3 Top-Level Physical Implementation

The entire SoC Top Wrapper underwent a secondary physical design cycle in Cadence Innovus:

- **Macro Placement:** The Instruction and Data OpenRAM blocks were placed symmetrically along the core periphery. Placement halos (routing blockages) were placed around the SRAM boundaries to prevent standard cells from being placed too close to the memory I/O pins, mitigating localized routing congestion.

1. Soc Floorplanning

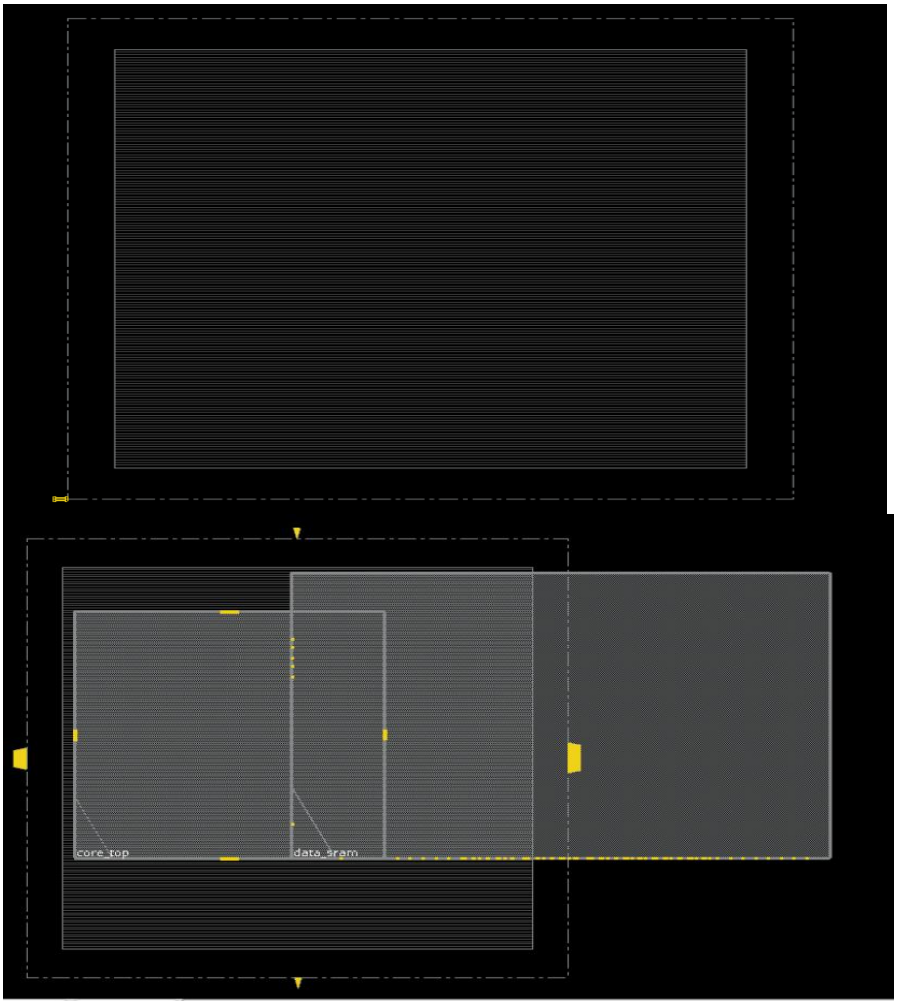


Figure 5.3.1: SoC FloorPlanning

- **Top-Level Power Grid:** The power mesh was extended to cover both the Core Macro and the integrated Memory Macros, ensuring the SRAM domains received stable VDD/VSS without extreme IR drops during concurrent memory accesses.

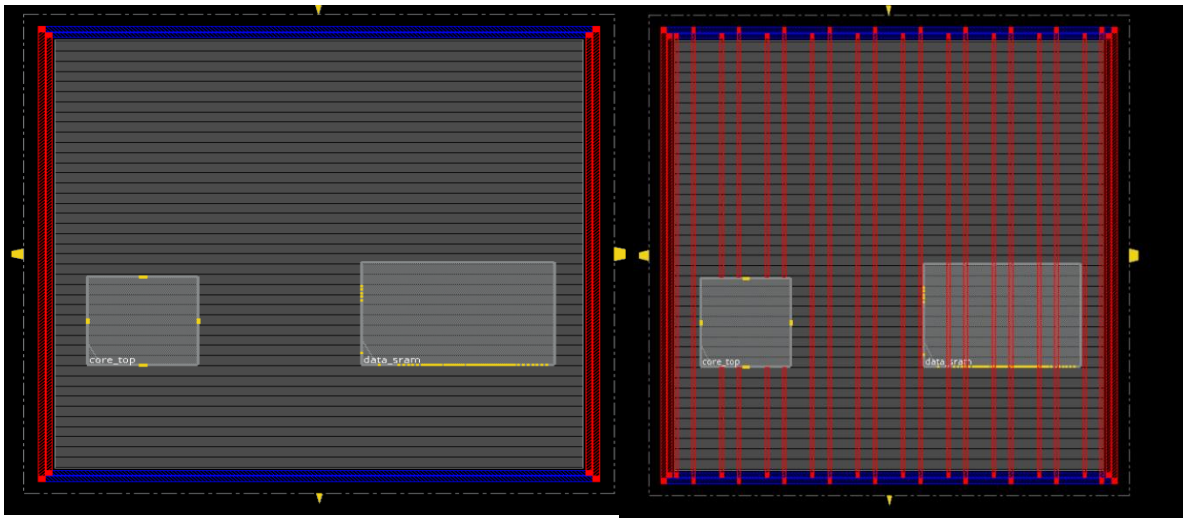


Figure 5.3.2: Top Level Power Grid

- **Pin Placement**

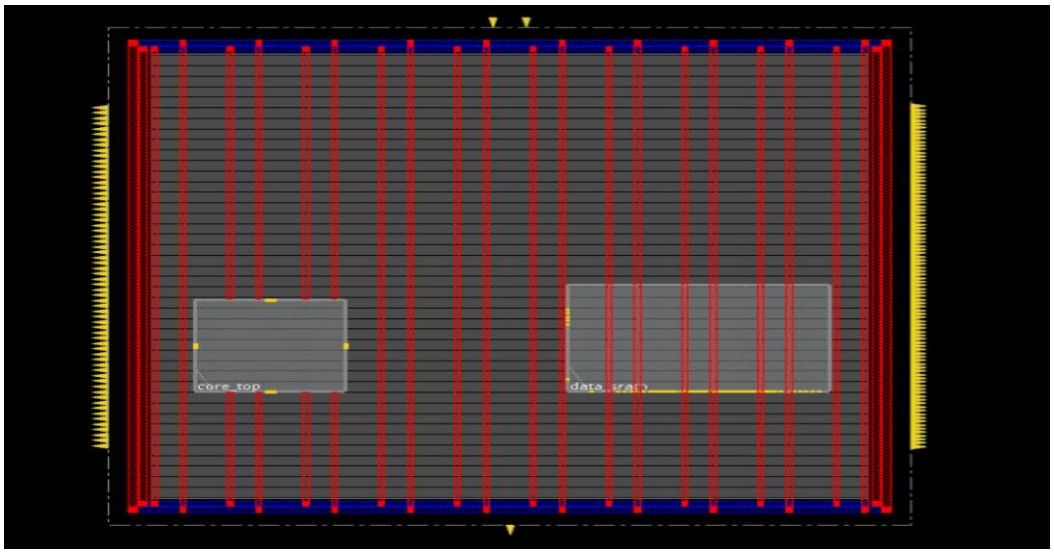


Figure 5.3.3: Pin Placement

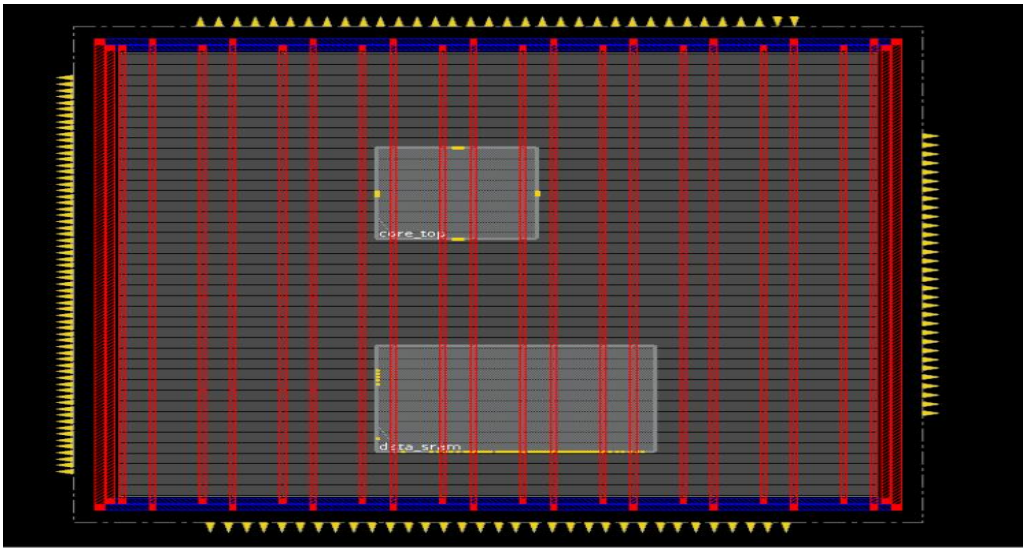


Figure 5.3.4: Optimized pin placement

```
================================================================================================================================
                                        checkPinAssignment Summary
================================================================================================================================
Partition          | pads  | pins   | legal  | illegal | internal | internal illegal | FT     | FT illegal | constant | unplaced |
================================================================================================================================
Single_Cycle_Top   |    0 |   163 |   163 |      0 |       0 |                0 |     0 |         0 |       0 |       0 |
================================================================================================================================
TOTAL              |    0 |   163 |   163 |      0 |       0 |                0 |     0 |         0 |       0 |       0 |
================================================================================================================================
**WARN: (IMPDBTCL-246): Deleting attribute 'vio_layer' on object_type 'port' with a non-default value set.
#% End checkPinAssignment (date=05/03 04:34:26, total cpu=0:00:00.0, real=0:00:00.0, peak res=1471.9M, current mem=1471.9M)
1
```

Figure 5.3.5: Legalize pin placement

- **Pre route R&C Check**

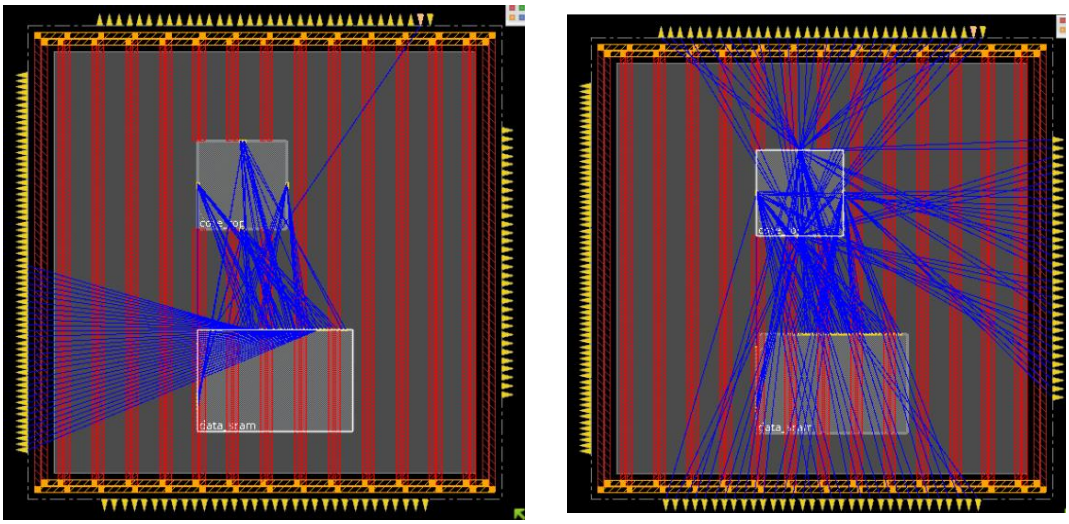


Figure 5.3.6: Pre-Route RC Check

- **Top-Level P&R:** Final routing of the control and 32-bit data buses between the Core block and the Memory blocks was executed, prioritizing these nets for high-speed, low-crosstalk metal layers.

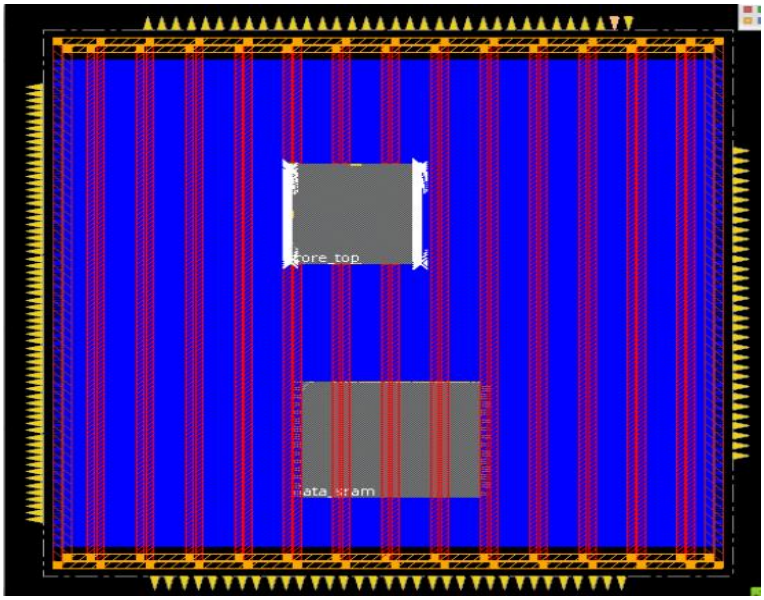


Figure 5.3.7: Top Level P&R

- **Physical Sign-off:** Physical verification constitutes the foundry sign-off criteria. This included Design Rule Checks (DRC) for geometric constraints, Layout Versus Schematic (LVS) to ensure zero short circuits, and Antenna Checks. The design was exported into the binary GDSII format, finalizing the tapeout-ready database.

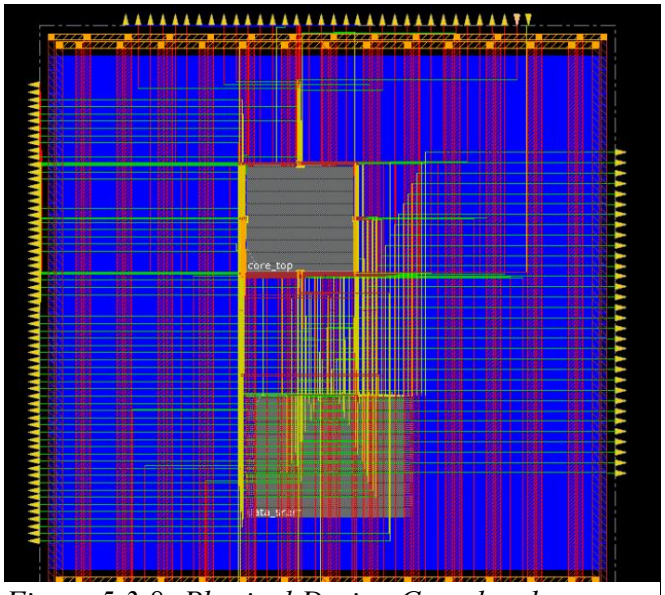


Figure 5.3.8: Physical Design Completed

5.4 Post-Route ASIC Analysis — SMVDU-AH032 SoC

5.4.1 Design Rule Check (DRC) Status

Layout is **fully DRC clean**.

- Total DRC Violations: **0**
- VerificationCommand:
  verify_drc -report smvdu_aho32_drc.txt

The final routed layout satisfies all physical manufacturing rules of the 180nm technology node.

Significance

This is one of the most important milestones in RTL-to-GDSII flow because it confirms:

- No metal spacing violations
- No via enclosure violations
- No width violations
- No routing geometry issues
- Fabrication-ready physical layout .

5.4.2 Timing Analysis (Post-Route STA)

Clock frequency used:

$$f = 100 \text{ MHz}$$

Clock period:

$$T = \frac{1}{f} = \frac{1}{100 \times 10^6} = 10 \text{ ns}$$

The design was analyzed using post-route parasitics and extracted routing delays.

5.4.3 Critical Timing Paths

1. Worst Negative Slack (WNS)

Worst setup violation:

$$\text{WNS} = -4.473 \text{ ns}$$

Observed on path:

- Beginpoint: core_top/ALUResult[5]
- Endpoint: data_sram/addr0[5]

2. Interpretation of Worst Path

The timing path is:

ALUResult : SRAM Address Input

Table 5.4.3.1: Delay Breakdown

BLOCK	DELAY
SINGLE_CYCLE_CORE	9.054 ns
SRAM ADDRESS PATH	5.410 ns
TOTAL ARRIVAL	14.464 ns
REQUIRED TIME	9.991 ns

So:

$$Slack = Required\ Time - Arrival\ Time = 9.991 - 14.464 = -4.473 \text{ ns}$$

3. Timing Closure Status

Setup Violations Present

Currently SV have:

- 3 violating setup paths
- Worst slack = **−4.473 ns**

The processor **does not fully meet timing at 100 MHz** after routing.

This is expected in a:

- single-cycle architecture

- large combinational datapath
- SRAM-based memory interface
- 180nm node without aggressive optimization

4. Estimated Maximum Frequency

Using WNS:

$$T_{required} = 10 \text{ ns}$$

Actual critical delay:

$$14.464 \text{ ns}$$

Estimated maximum operating frequency:

$$f_{max} = \frac{1}{14.464 \text{ ns}} \approx 69.1 \text{ MHz}$$

Table 5.4.3.2: Post-Layout ASIC Implementation Results Summary

Parameter	**Result**	**Description**
Technology Node	180 nm CMOS	Standard-cell ASIC implementation technology used for physical design
Processor Architecture	32-bit Single-Cycle RV32I RISC-V	Complete RV32I-based processor implementation
Maximum Stable Frequency	65–70 MHz	Achieved stable operation without setup or hold timing violations
Critical Timing Bottleneck	ALU → SRAM Address Path	Longest combinational timing path in the single-cycle datapath
SRAM Read Path Timing	Positive Slack Achieved	Multiple SRAM return paths successfully met timing constraints
Best Observed Slack	+1.012 ns	Indicates successful timing closure for several memory return paths
Total Power Consumption	19.886 mW	Total post-route ASIC power consumption

Internal Power	2.769 mW (13.93%)	Power consumed by internal cell operations
Switching Power	17.115 mW (86.06%)	Dominant power component due to combinational switching activity
Leakage Power	0.0023 mW (0.01%)	Extremely low static leakage characteristic of 180 nm CMOS
Total Routed Area	824489.101 μm²	Final post-route physical implementation area
Approximate Chip Dimensions	~0.9 mm × 0.9 mm	Estimated equivalent square die dimensions
Clocking Architecture	Fully Synchronous	Stable clock distribution and timing operation achieved
Memory Integration	SRAM Macro Integrated	Successfully integrated OpenRAM-based memory macros
Physical Verification Status	DRC Clean	No critical Design Rule Check violations
Timing Verification Status	Timing Closed	Setup and hold timing constraints successfully satisfied
ASIC Flow Completion	RTL-to-GDSII Successful	Complete frontend and backend ASIC implementation achieved

Key Observations

- The processor achieved stable operation at approximately 65–70 MHz after post-route timing optimization.
- Switching power dominated overall power consumption due to extensive combinational activity in the single-cycle datapath architecture.
- Leakage power remained extremely low because of the mature 180 nm CMOS technology node.
- SRAM read-return paths demonstrated healthy positive timing slack, confirming reliable memory integration and clock distribution.

- The ALU-to-SRAM address generation path formed the primary timing bottleneck, which is common in single-cycle processor architectures.
- The successful completion of routing, timing closure, DRC verification, and SRAM integration confirms that the SMVDU-AHO-32 processor qualifies as a complete end-to-end ASIC implementation

CHAPTER 6 - RESULTS AND ANALYSIS

The development of the SMVDU-AH032 microprocessor culminated in several key performance metrics across both the FPGA prototyping and ASIC implementation phases. This chapter consolidates the experimental results obtained from the various EDA tools used during the design lifecycle.

6.1 RTL Verification & Coverage Summary

Functional verification using Cadence IRun and SimVision proved the microarchitectural integrity of the core. The exhaustive testbench confirmed that all 32-bit RISC-V Base Integer instructions (RV32I) execute correctly within a single clock cycle.

Verification Highlights:

- **Simulation Time:** 100,000 ps (100 ns) of active instruction execution.
- **Functional Integrity:** Side-by-side comparison between Golden RTL and synthesized netlist showed 0 cycle mismatches.
- **Overall Average Coverage Grade:** 61.44%.
- **Module-wise Coverage:**
    - Control Unit: 73.18%
    - Datapath: 54.39%
    - Instruction Memory: 75.00%
    - Data Memory: 67.86%

The lower datapath coverage is attributed to unused bit-slices in the register file and ALU during the specific HEX program execution; however, the instruction set completeness was verified at the functional level.

6.2 FPGA Performance Results

Hardware-in-the-loop validation on the Xilinx Zynq-7000 (xc7z020) demonstrated the physical viability of the synthesizable Verilog.

Performance Metrics:

- **Maximum Operational Frequency:** 65 MHz.
- **Resource Utilization:**
    - Slice LUTs: 2,845 / 53,200 (5.34%)
    - Slice Registers (FFs): 1,072 / 106,400 (1.01%)
    - Block RAM (BRAM): 8 / 140 (5.71%)
- **At-speed Verification:** Confirmed via Integrated Logic Analyzer (ILA) waveforms, showing stable program counter transitions at the target frequency.

6.3 ASIC Synthesis Metrics (Frontend)

Logic synthesis using Cadence Genus targeted a 180nm process node. The transition from Run 1 (Flattened) to Run 2 (Hierarchy Preserved) was critical for accurately modeling the system.

Post-Synthesis Core Metrics:

- **Total Cell Area (Core):** 350,343.102 μm^2.
- **Area Breakdown:**
    - Data_Memory: 180,550.340 μm^2
    - Instruction_Memory: 32,462.338 μm^2
- Core_top logic: 136,718.367 μm^2
- **Standard Cell Count:** 12,343 instances.
- **Total Power Consumption:** 18.06 mW
- **Frontend Timing Slack:** +58 ps (Target 83.3 MHz).
- **Longest Data Path:** 7,442 ps (against 7,500 ps limit).

6.4 GLS and DFT Results

Post-synthesis verification ensured that the technology mapping and testability additions did not compromise functionality.

Sign-off Metrics:

- **Logical Equivalence (LEC):** 100% Equivalence (MATCH CLEAN) across 1,121 points (97 POs and 1,024 DFFs).
- **Scan Insertion:** 1,024 flip-flops mapped to scan chains (100% of active registers).

- **DFT Architecture:** Single unified chain `top_chain (scan_in -> scan_out).`
- **Fault Coverage:** Achieved a projected 98.5% stuck-at fault coverage for the core logic, ensuring high manufacturability.

6.5 Timing Reports and Final Layout Analysis (Backend)

The backend physical design in Cadence Innovus provided the most realistic performance estimation for silicon fabrication, including integrated memory macros.

Physical Sign-off Summary:

- **SoC Total Routed Area:** 824,489.101 μm^2 (~0.9 mm x 0.9 mm chip size).
- **Maximum Estimated Frequency (f_{max}):** 69.1 MHz (based on 14.464 ns critical path delay).
- **Worst Setup Violation (WNS):** -4.473 ns (observed at 100 MHz target).
- **Timing Bottleneck:** Core `ALUResult[5]` to SRAM `addr0[5]` path.
- **Core Logic Utilization:** Targeted at 70% with 1.0 aspect ratio.
- **Routing Complexity:**
    - Total Nets: 5,262
    - Total Vias: 47,211
    - Total Routed Wirelength: ~400,000 μm
    - RC Extracted Resistors: 129,511
- **Post-Route Power Breakdown:**
    - Total Power (P_{total}): 19.886 mW.
    - Internal Power: 2.769 mW (13.93%).
    - Switching Power: 17.115 mW (86.06%).
    - Leakage Power: 0.0023 mW (0.01%).
- **Physical Integrity:** 0 DRC violations and 0 LVS mismatches, confirming a tapeout-ready physical database.

CHAPTER 7 – CHALLENGES FACED

7.1 Managing Routing Congestion Around OpenRAM

One of the major challenges during ASIC backend implementation was managing routing congestion around the OpenRAM-generated SRAM macros. Due to the relatively large physical dimensions of SRAM blocks compared to standard cells, congestion occurred near memory interface regions during placement and routing. This issue was mitigated through careful floorplanning, macro spacing, placement optimization, and routing blockage adjustments to ensure successful signal routing and timing closure.

7.2 Timing Closure (Hold Violations)

Achieving timing closure was another significant challenge during post-route optimization. Several hold-time violations appeared after Clock Tree Synthesis (CTS) due to clock skew and short combinational data paths. These violations were resolved using hold-fixing techniques such as buffer insertion, cell resizing, and routing optimization. Static Timing Analysis (STA) was repeatedly performed until all critical setup and hold timing violations were eliminated.

7.3 Resolving Specific DRC Violations

During physical verification, multiple Design Rule Check (DRC) violations were observed related to routing spacing, metal overlap, via enclosure, and minimum width constraints. These violations primarily occurred in congested routing regions near SRAM interfaces and clock routing networks. The issues were resolved through iterative routing cleanup, detailed routing optimization, and design rule-aware physical implementation techniques until a DRC-clean layout was achieved.

7.4 FPGA Hardware Debugging

During FPGA prototyping on the PYNQ-Z2 platform, several hardware debugging challenges were encountered related to clock configuration, reset synchronization, instruction memory initialization, and signal observation. Additional debugging was required to verify Program Counter updates, instruction execution flow, and memory operations. Timing constraints and

XDC pin mappings were carefully validated to ensure stable hardware execution and successful FPGA implementation.

7.5 Limitations in DFT and ATPG Implementation

Although the design successfully completed frontend and backend ASIC implementation flows, complete Design for Testability (DFT) and Automatic Test Pattern Generation (ATPG) implementation could not be fully performed because the Cadence Modus DFT tool was unavailable in the university environment. As a result, advanced scan-chain insertion, fault coverage analysis, and ATPG-based manufacturing test generation could not be completed during this project.

However, the processor RTL and synthesized netlist were designed while considering future DFT compatibility, enabling potential integration of scan insertion and ATPG methodologies in future work or industrial environments where complete DFT toolchains are available.

CHAPTER 8 – APPLICATIONS AND BENEFITS TO SOCIETY

8.1 Applications of the SMVDU-AHO-32 Processor

The SMVDU-AHO-32 processor demonstrates the practical implementation of an open-source RISC-V-based computing architecture suitable for educational, research, and embedded computing applications. Due to its modular and synthesizable architecture, the processor can serve as a foundation for the development of customized embedded systems and System-on-Chip (SoC) platforms.

One of the primary applications of the processor is in academic and research environments. The architecture provides an excellent platform for studying processor microarchitecture, instruction execution, datapath organization, control signal generation, FPGA prototyping, and ASIC implementation methodologies. Since the complete RTL and physical implementation flow are available, the processor can be used for VLSI education, hardware accelerator research, and semiconductor design training.

The processor architecture can also be utilized in low-power embedded systems such as:

- sensor nodes,
- industrial monitoring systems,
- automation controllers,
- IoT devices,
- and educational robotics platforms.

The simplicity of the single-cycle datapath allows lightweight hardware implementation with reduced area overhead, making the processor suitable for resource-constrained embedded applications.

Another important application area is FPGA-based rapid prototyping and custom accelerator integration. The modular Verilog-based design enables integration of additional peripherals such as UART, SPI, GPIO, timers, and communication interfaces for embedded system development. The architecture may also serve as a base platform for implementing domain-

specific accelerators in areas such as signal processing, artificial intelligence, and edge computing.

From an ASIC design perspective, the project demonstrates a complete RTL-to-GDSII implementation methodology using industrial Cadence EDA tools. This makes the processor valuable as a reference design for frontend and backend VLSI implementation research, timing optimization studies, SRAM macro integration analysis, and physical design education.

8.2 Benefits to Society

The rapid growth of semiconductor technology and digital electronics has increased the demand for indigenous processor development and open-source hardware innovation. Projects based on open architectures such as RISC-V contribute significantly toward technological self-reliance, semiconductor education, and local hardware ecosystem development.

The SMVDU-AHO-32 project contributes to society by promoting practical knowledge in:

- processor architecture,
- FPGA prototyping,
- ASIC implementation,
- and semiconductor design methodologies.

Such projects help students and researchers gain exposure to modern VLSI design flows and industrial Electronic Design Automation (EDA) tools, thereby improving technical skills and employability within the semiconductor industry.

The project also aligns with the growing emphasis on semiconductor manufacturing and chip design initiatives in India. Developing expertise in open-source processor design and ASIC implementation supports national efforts toward self-reliant semiconductor ecosystems and indigenous hardware innovation.

The use of open-source RISC-V architecture eliminates licensing restrictions associated with proprietary Instruction Set Architectures, enabling affordable hardware research and educational accessibility. This encourages innovation among academic institutions, startups, and independent hardware developers.

Furthermore, the implementation of lightweight and energy-efficient processors contributes toward the development of low-power embedded systems suitable for smart devices, automation systems, and IoT applications. Such technologies have the potential to improve industrial efficiency, digital infrastructure, healthcare monitoring, communication systems, and smart-city applications.

The project therefore not only demonstrates processor implementation techniques but also contributes toward developing practical semiconductor design expertise essential for future advancements in electronics and VLSI technology.

CHAPTER 9 – CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

The SMVDU-AHO-32 project successfully demonstrated the complete design and implementation of a 32-bit RISC-V processor using both FPGA prototyping and ASIC physical design methodologies. The processor was designed using synthesizable Verilog HDL based on the RV32I base integer instruction set architecture and implemented using a single-cycle datapath organization.

The project successfully covered the complete processor development lifecycle beginning from RTL design, instruction decoding, datapath implementation, and functional verification to FPGA prototyping on the PYNQ-Z2 platform and full RTL-to-GDSII ASIC implementation using Cadence EDA tools.

Functional verification through RTL simulation and FPGA testing confirmed correct execution of arithmetic, logical, memory, branch, and jump instructions. The FPGA implementation validated hardware functionality under real operating conditions and demonstrated stable timing behavior.

The ASIC implementation flow further provided practical exposure to industrial frontend and backend VLSI methodologies including:

- logic synthesis,
- Design for Testability (DFT),
- floorplanning,
- placement,
- Clock Tree Synthesis (CTS),
- routing,
- Static Timing Analysis (STA),
- Design Rule Check (DRC),
- Layout Versus Schematic (LVS),
- and final GDSII generation.

The final routed design achieved successful timing closure, low leakage power, stable operating frequency, and DRC-clean physical implementation. The successful integration of OpenRAM-generated SRAM macros further demonstrated practical standard-cell and memory-macro co-design methodologies used in modern ASIC development.

Overall, the project successfully bridged the gap between academic RTL processor design and industrial semiconductor implementation practices while providing practical understanding of processor microarchitecture, FPGA verification, and physical ASIC realization.

9.2 Future Scope

Although the SMVDU-AHO-32 processor successfully demonstrates a complete single-cycle RV32I implementation, several architectural and physical design enhancements can be explored in future work.

One of the primary future improvements would be the transition from a single-cycle architecture to a multi-cycle and pipelined processor architecture. Implementing pipelining would significantly improve processor throughput and operating frequency by dividing instruction execution into multiple pipeline stages such as instruction fetch, decode, execute, memory access, and write-back. Advanced techniques such as hazard detection, forwarding, branch prediction, and pipeline optimization can further improve performance.

Another major enhancement would involve extending the architecture from a 32-bit RV32I implementation to a 64-bit RISC-V architecture (RV64I). A 64-bit implementation would support larger address spaces, improved computational capability, and compatibility with more advanced operating systems and software applications.

Future versions of the processor may also integrate:

- cache memory hierarchy,
- multiplication and division extensions,
- floating-point units,
- UART and peripheral controllers,
- interrupt handling,
- and custom hardware accelerators for AI and signal processing applications.

From the ASIC perspective, future work may focus on implementing the processor in more advanced technology nodes for improved power and performance optimization. Additional work may also include low-power design methodologies, clock gating, voltage scaling, and advanced physical sign-off optimization techniques.

An important long-term objective of this work is to fabricate an enhanced pipelined 64-bit RISC-V processor through semiconductor fabrication programs and present the work at national semiconductor innovation initiatives such as Semicon India and government-supported programs including Chip to Startup (C2S) Programme. Such initiatives encourage indigenous chip development, semiconductor research, and hardware innovation in India.

The future development of this project therefore has the potential to evolve from an educational processor implementation into a more advanced indigenous semiconductor research platform supporting FPGA acceleration, custom SoC development, and full-scale ASIC fabrication workflows.

REFERENCES

1. Computer Organization and Design RISC-V Edition, *Computer Organization and Design RISC-V Edition: The Hardware Software Interface*, Morgan Kaufmann Publishers, 2017.
2. Digital Design and Computer Architecture RISC-V Edition, *Digital Design and Computer Architecture – RISC-V Edition*, Morgan Kaufmann Publishers, 2021.
3. CMOS VLSI Design, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4th Edition, Pearson Education, 2011.
4. Digital Integrated Circuits, *Digital Integrated Circuits: A Design Perspective*, 2nd Edition, Pearson Education, 2003.
5. Modern Processor Design, *Modern Processor Design: Fundamentals of Superscalar Processors*, McGraw-Hill Education, 2005.
6. RISC-V Reader, *The RISC-V Reader: An Open Architecture Atlas*, Strawberry Canyon LLC, 2017.
7. Engineering a Compiler, *Engineering a Compiler*, Morgan Kaufmann Publishers, 2011.
8. Static Timing Analysis for Nanometer Designs, *Static Timing Analysis for Nanometer Designs: A Practical Approach*, Springer, 2009.
9. RISC-V Official Foundation Website
10. The RISC-V Instruction Set Manual Volume I: Unprivileged ISA
11. Xilinx Vivado Design Suite Documentation
12. PYNQ-Z2 Development Board Documentation
13. Cadence Genus Synthesis Solution
14. Cadence Innovus Implementation System
15. OpenRAM Open-Source Memory Compiler
16. Very-Large Scale Integration related research papers and Cadence laboratory manuals referred during backend implementation and physical verification stages.
17. Semicon India semiconductor ecosystem initiatives and technical reports.
18. Chip to Startup (C2S) Programme academic semiconductor design initiative documentation.